

Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster en Big Data y Computación en la Nube

Diseño de una arquitectura lakehouse escalable para la integración y análisis de información curricular académica

Carlos Martínez Jaén

Octubre, 2026

Escuela Superior de Ingeniería Informática Albacete

Escuela Superior de Informática

TRABAJO FIN DE MÁSTER

Máster en Big Data y Computación en la Nube

Diseño de una arquitectura lakehouse escalable para la integración y análisis de información curricular académica

Autor: Carlos Martínez Jaén

Director: Luis De La Ossa Jiménez

UCLM Universidad Castilla-La Mancha

Octubre, 2026

Por la libertad al final
camino

Declaración de autoría

Yo, Carlos Martínez Jaén , con DNI 00000000X , declaro que soy el único autor del trabajo fin de máster titulado “Diseño de una arquitectura lakehouse escalable para la integración y análisis de información curricular académica ”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a 5 de octubre de 2026

Fdo.: Carlos Martínez Jaén

Resumen

La información curricular y de trayectoria investigadora se reparte entre documentos individuales, como los currículos CVN, y registros públicos a escala masiva, como los perfiles de ORCID. Los sistemas de información de investigación interoperan datos de este tipo sobre infraestructura institucional cerrada, y no existe una plataforma abierta, autoalojada y reproducible que combine ambas naturalezas de dato y las procese de forma conjunta.

Este Trabajo Fin de Máster diseña, despliega y evalúa una plataforma *lakehouse* que integra ORCID real con CVN sintético y publica indicadores de investigación. La plataforma se despliega sobre Kubernetes y combina almacenamiento de objetos con Iceberg, procesamiento distribuido con Spark, orquestación con Airflow, publicación en PostgreSQL y visualización con Superset. Los currículos CVN se generan de forma sintética, válidos frente al formato Open CVN y sembrados con campos públicos de ORCID, de modo que no se tratan datos personales de terceros. Esa siembra enlaza cada currículo con su perfil real y sirve de base a una resolución de identidad determinista, con evidencia explícita en cada fusión.

La evaluación usa datos reales. Sobre más de 26 millones de registros de ORCID se filtraron 301 763 perfiles con afiliación española, y la resolución de identidad alcanzó una precisión del 95,8% y una cobertura del 74,2% en su regla de repliegue por nombre y afiliación. Un banco de pruebas de rendimiento sobre tres volúmenes de datos demuestra que el número de ejecutores de Spark es una palanca de fiabilidad, no solo de velocidad: con la memoria fija por ejecutor, un único ejecutor deja de completar la transformación a partir del doble del volumen base. Por último, la plataforma completa se reconstruyó desde cero sobre un clúster aislado siguiendo únicamente una guía escrita, lo que confirma su reproducibilidad.

Agradecimientos

Tras mucho esfuerzo y dedicación, ha llegado el final de este camino estudiantil. Después de 5 largos y maravillosos años, me despido de la institución que, junto a mis amigos y familia, me ha visto crecer y madurar como ingeniero y persona. Me llevaré muy buenos recuerdos de esta etapa de mi vida; sin embargo, tengo muchas ganas de ver qué me depara el mundo más allá del ámbito académico. He de agradecer a todos los que me han acompañado durante todo este tiempo, en especial a mi pareja, mejor amiga y compañera de vida, Mapi, por todo lo que hemos pasado juntos y lo que queda.

Finalmente quiero agradecer a mis padres, que me han apoyado en todo momento y me han dado la oportunidad de estudiar y crecer como persona. Sin ellos, nada de esto hubiera sido posible.

Índice general

1 Introducción, motivación y objetivos 1

1.1 Contexto y continuidad con el Trabajo Fin de Grado 1

1.2 Problema de partida y motivación 2

1.3 Objetivos 3

1.4 Aspectos éticos y de privacidad en el tratamiento de datos 5

1.5 Resultados de aprendizaje del máster 6

1.6 Estructura del documento 6

2 Estado del arte y decisiones arquitectónicas 9

2.1 De los sistemas de almacenamiento analítico al lakehouse 9

2.2 Formato de tabla: Apache Iceberg frente a sus alternativas 10

2.2.1 Catálogo de Iceberg 11

2.3 Almacenamiento de objetos: MinIO frente a sus alternativas 12

2.4 Sustrato de orquestación de contenedores y entorno de despliegue 12

2.5 Procesamiento distribuido: Apache Spark frente a sus alternativas 13

2.6 Orquestación de flujos ETL: Apache Airflow frente a sus alternativas 14

2.7 Capa de consulta interactiva y capa de visualización 15

2.8 Síntesis y arquitectura resultante 16

3 Infraestructura: despliegue del clúster y servicios base 19

3.1 Arquitectura de despliegue 19

3.2 Clúster Kubernetes local 20

3.3 Servicios base: MinIO, PostgreSQL y Airflow 21

3.4 Catálogo Iceberg sobre MinIO 21
3.5 Ejecución de Spark desde Airflow 23
3.6 Verificación de extremo a extremo 24

4 Ingesta y fusión de fuentes heterogéneas 25

4.1 Arquitectura de la ingesta 25
4.2 Adquisición de datos de ORCID 26
4.3 Generación de currículos CVN sintéticos 28
4.4 Aterrizaje en bronze con procedencia 29
4.5 Verificación de extremo a extremo 30

5 Procesamiento distribuido: transformación, resolución de entidades e indicadores 33

5.1 Arquitectura de la transformación 33
5.2 Validación y normalización de bronze a silver 34
5.3 Resolución de identidad entre fuentes 35
5.4 Cálculo de indicadores y publicación en PostgreSQL 37
5.5 Verificación de extremo a extremo 38

6 Visualización, evaluación de rendimiento y endurecimiento 41

6.1 Panel de indicadores 41
6.2 Metodología y resultados del banco de pruebas de rendimiento 43
6.3 Endurecimiento: fallos operativos reales encontrados y corregidos 45
6.4 Verificación de extremo a extremo: reconstrucción desde cero 46

7 Conclusiones, competencias y trabajo futuro 49

7.1 Resumen del trabajo 49
7.2 Cumplimiento de los objetivos 50
7.3 Contribuciones principales 50
7.4 Resultados de aprendizaje desarrollados 51
7.5 Limitaciones y trabajo futuro 52
7.6 Conclusiones 53

Referencia bibliográfica 58

Anexos 59

Anexo A: Guía de reproducibilidad de la plataforma 61
A.1 Prerrequisitos y versiones fijadas 61
A.2 Clúster y servicios base 62
A.3 Imágenes de Spark y de ingesta 63
A.4 Ejecución de los flujos 64
A.5 Panel de indicadores y verificación 64
A.6 Recuperación de los fallos conocidos 65
Anexo B: Resultados completos del banco de pruebas 67
B.1 Volumen de datos por escala 67
B.2 Tiempos, aceleración y eficiencia por trabajo 68
B.3 Ejecuciones excluidas 70
B.4 Corrección del resultado entre configuraciones 70
Anexo C: Modelo de datos de la plataforma 71
C.1 Organización del almacenamiento 71
C.2 Sobre de procedencia de bronze 71
C.3 Tablas de silver 72
C.4 Tablas de gold y su espejo en PostgreSQL 73
Anexo D: Resolución de identidad en detalle 75
D.1 Normalización previa a la comparación 75
D.2 Compatibilidad de nombres 75
D.3 Evidencia guardada en cada enlace 76
D.4 Umbral de similitud entre organizaciones 76
D.5 Análisis de errores y decisión final 77
D.6 Alternativas más estrictas 77
D.7 Cobertura por variante de nombre 78
D.8 Método de evaluación 78
Anexo E: Registro de limitaciones 81
E.1 Infraestructura y cadena de suministro 81
E.2 Alcance de los datos y de la ingesta 83
E.3 Resolución de identidad e indicadores 83

E.4 Escalabilidad y medición 84

Anexo F: Repositorio del proyecto y guía de uso 85

Índice de figuras

3.1 Cadena de capas de la plataforma. 20
3.2 Separación, dentro del mismo cubo, entre el prefijo de aterrizaje crudo y las tablas Iceberg del catálogo. 22

4.1 Orden de tareas del flujo `ingest_validate`. 26

5.1 Orden de aplicación de las reglas de resolución de identidad. 36

6.1 Panel de indicadores de investigación en Apache Superset, con los tres indicadores y la tabla de procedencia de la ejecución. 42
6.2 Tiempo de ejecución y aceleración del trabajo de validación y normalización, sobre el volumen de datos base. 44

7.1 Tiempo de ejecución y aceleración del cálculo de indicadores. 69

Índice de tablas

1.1 Relación entre objetivos específicos y capítulos del documento. 5
1.2 Relación entre resultados de aprendizaje del máster y capítulos del documento. 6

2.1 Comparación de formatos de tabla abierta para arquitecturas *lakehouse*. . . 11
2.2 Síntesis de las decisiones arquitectónicas de la plataforma. 16

3.1 Servicios desplegados en el clúster local. 20

4.1 Los dos mecanismos públicos de adquisición de datos de ORCID. 27
4.2 Campos del sobre de procedencia, comunes a las tres fuentes. 30

5.1 Resolución de identidad sobre la ejecución de 10 000 documentos CVN. . . 37

6.1 Exponente de crecimiento del tiempo de ejecución frente al volumen de datos, por trabajo y número de ejecutores. 45

7.1 Grado de cumplimiento de los objetivos específicos. 50
7.2 Resultados de aprendizaje del máster y evidencia concreta de cada uno. . . 51
7.3 Limitaciones documentadas y trabajo futuro asociado. 53
7.4 Versiones fijadas de los componentes desplegados. 62
7.5 Datos aterrizados en cada escala del banco de pruebas. 67
7.6 Validación y normalización de bronze a silver. 68
7.7 Cálculo de indicadores de silver a gold. 68
7.8 Publicación de gold en PostgreSQL. 69
7.9 Comprobación de igualdad de contenido entre configuraciones. 70
7.10 Campos del sobre de procedencia de bronze. 72
7.11 Tablas de silver. 73
7.12 Tablas de gold publicadas en PostgreSQL. 74
7.13 Efecto del umbral de similitud entre organizaciones. 77
7.14 Variantes más estrictas de la regla de repliegue. 78
7.15 Cobertura de la regla de repliegue por variante de nombre. 78

Índice de algoritmos

Índice de listados de código

A.1 Instalación del clúster y de los tres servicios base. 63
A.2 Construcción e importación de las imágenes de la plataforma. 63
A.3 Entrega y lanzamiento de los dos flujos de Airflow. 64
A.4 Verificación del repositorio y del estado del clúster. 65

1. Introducción, motivación y objetivos

El presente capítulo introduce el contexto en el que se enmarca este Trabajo Fin de Máster, en adelante TFM, describe la motivación que justifica su desarrollo y delimita los objetivos que guían la solución propuesta. Para ello, en primer lugar se sitúa el trabajo con respecto al Trabajo Fin de Grado, en adelante TFG, previo del mismo autor, del que parte directamente. A continuación, se expone el problema de partida y la motivación que lo sustenta, se establecen los objetivos general y específicos, se recogen los aspectos éticos y de privacidad que afectan al tratamiento de los datos empleados y se relacionan los objetivos con los resultados de aprendizaje del máster. Finalmente, se presenta la estructura seguida por el resto del documento.

1.1. Contexto y continuidad con el Trabajo Fin de Grado

Este TFM se construye directamente sobre el trabajo ya cerrado y defendido en el TFG del mismo autor [Jaén, 2026], del que hereda el punto de partida técnico: una capa estructural reproducible generada a partir de los esquemas oficiales de CVN, una normalización trazable de sus metadatos, y un formato abierto, validable y extensible, Open CVN JSON, con su propio contrato de parseo y validación.

El presente trabajo reutiliza ese resultado como una de sus dos fuentes de datos, no como objeto de estudio: se reutilizan directamente el esquema JSON Schema de Open CVN, su contrato de validación ya implementado, y las convenciones de trazabilidad de procedencia, en inglés *provenance*, establecidas en la capa semántica del TFG. Por ello, este documento no vuelve a definir qué es CVN ni qué es Open CVN, remitiendo esa exposición al TFG anterior, y se limita a describir cómo dicho formato, ya cerrado, se ingiere, valida y procesa dentro de la plataforma distribuida que aquí se construye.

Este cambio de perspectiva, de un formato y una herramienta de un único usuario a una plataforma que ingiere, fusiona y procesa datos a escala, responde también a un cambio de programa académico. Mientras que el TFG se enmarcaba en la intensificación de Computación, este TFM pertenece al Máster en Big Data y Computación en la Nube, cuyos resultados de aprendizaje se detallan en la Sección 1.5.

1.2. Problema de partida y motivación

La información curricular y de trayectoria investigadora se encuentra dispersa en fuentes heterogéneas, de distinta naturaleza y de distinto volumen. Por un lado, documentos curriculares como CVN describen la actividad de un investigador de forma estructurada pero unitaria, pensada para su presentación individual ante convocatorias, evaluaciones o acreditaciones. Por otro lado, registros públicos como los perfiles de ORCID, siglas en inglés de *Open Researcher and Contributor ID* [ORCID, 2026b], agregan identificadores, filiaciones y producción científica de investigadores de todo el mundo a escala global. Solo el fichero público de datos de ORCID de 2025 reúne más de 26 millones de registros [ORCID, 2025].

Los sistemas de información de investigación, conocidos por sus siglas en inglés como CRIS, de *Current Research Information Systems*, interoperan datos de este tipo entre instituciones mediante modelos comunes como CERIF [euroCRIS, 2026] o plataformas como VIVO [VIVO Project, 2026]. Sin embargo, lo hacen sobre infraestructura institucional cerrada, sin publicar una plataforma de referencia abierta, autoalojada y reproducible que combine específicamente currículos normalizados como CVN con perfiles públicos como ORCID. Estas dos naturalezas de dato, documental y de registro masivo público, rara vez se integran, procesan conjuntamente y publican como indicadores agregados dentro de una misma plataforma de este tipo.

Esta ausencia responde a un problema de arquitectura. Integrar fuentes de esta naturaleza exige:

- **Almacenamiento escalable**, capaz de crecer con el volumen real de ORCID.
- **Un formato de tabla que preserve evolución de esquema y trazabilidad**, dado que CVN y ORCID no comparten esquema.
- **Procesamiento distribuido tolerante a fallos**, para transformar ese volumen en un tiempo razonable.
- **Orquestación de flujos de trabajo**, que encadene ingesta, transformación y publicación de forma reproducible.

- **Una capa de resolución de identidad**, capaz de fusionar registros de fuentes distintas que describen a la misma persona.

Ninguno de estos elementos formaba parte del alcance del TFG, centrado deliberadamente en un único documento curricular.

La motivación de este trabajo consiste, por tanto, en demostrar, mediante una plataforma real desplegada y no mediante un diseño exclusivamente documental, las competencias propias de un perfil de Big Data y Computación en la Nube:

- Levantar infraestructura distribuida.
- Ingerir y fusionar fuentes heterogéneas.
- Procesar los datos a escala con tolerancia a fallos.
- Evaluar el sistema con datos y cifras reales obtenidos de su propia ejecución, en lugar de resultados simulados o proyectados.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Máster es *diseñar, desplegar y evaluar una plataforma que integre datos curriculares de CVN sintético y datos de investigación de ORCID real, los procese de forma distribuida con resolución de identidad entre fuentes, y publique un conjunto reducido de indicadores de investigación en un panel de visualización, evaluando el sistema mediante un benchmark de rendimiento acotado*. Los objetivos que siguen se formulan de forma agnóstica respecto a la arquitectura y las tecnologías concretas que los satisfacen. Esa elección se presenta y se justifica frente a sus alternativas en el Capítulo 2.

Para alcanzar esta meta global se han definido los siguientes objetivos específicos:

1. **Desplegar la infraestructura base de la plataforma.** Comprende la puesta en marcha del entorno de cómputo y de los servicios de almacenamiento, persistencia y orquestación sobre los que se apoya el resto del sistema.
2. **Validar el mecanismo central de almacenamiento y procesamiento distribuido.** Antes de construir cualquier flujo de datos de negocio sobre la plataforma, se comprueba de forma aislada que dicho mecanismo funciona de extremo a extremo.
3. **Incorporar datos ORCID reales a la plataforma.** Combina el enriquecimiento puntual de registros concretos con la incorporación de un volumen real de datos, ambos obtenidos de fuentes públicas de ORCID.

4. **Incorporar currículos CVN a la plataforma, validados frente al formato Open CVN.** Estos currículos se obtienen por generación sintética a partir de datos públicos, no por recolección de currículos reales, tal y como se justifica en la Sección 1.4.
5. **Integrar los datos de ORCID con los datos CVN por medio del formato Open CVN, resolviendo la identidad de los registros que corresponden a la misma persona.** Constituye la fusión efectiva entre las dos fuentes de datos del sistema.
6. **Calcular indicadores de investigación agregados y publicarlos de forma consistente.** Los indicadores deben quedar disponibles para su consumo posterior sin estados intermedios ni publicaciones parciales.
7. **Visualizar los indicadores publicados en un panel accesible.** Permite comprobar que los indicadores calculados son consumibles más allá del propio sistema que los produce.
8. **Medir el efecto del grado de paralelismo del procesamiento distribuido sobre el tiempo de ejecución y la fiabilidad, a distintas escalas de datos.** Constituye el *benchmark* de rendimiento acotado exigido por el planteamiento del proyecto.
9. **Encontrar y corregir defectos de robustez operativa mediante el uso real del sistema completo.** Este objetivo, sin equivalente directo en el TFG anterior, exige documentar una guía de reproducción del entorno desde cero y registrar los fallos de infraestructura detectados y corregidos como parte de la evaluación del sistema.

La Tabla 1.1 relaciona estos objetivos con los capítulos principales en los que se desarrollan. Es en esos capítulos, no aquí, donde cada objetivo se concreta en una arquitectura y unas tecnologías determinadas.

Objetivo	Descripción resumida	Capítulos
OE1	Despliegue de la infraestructura base de la plataforma	3
OE2	Validación del mecanismo central de almacenamiento y procesamiento	3
OE3	Incorporación de datos ORCID reales	4
OE4	Incorporación de currículos CVN, validados frente a Open CVN	4
OE5	Integración de ORCID con CVN por medio de Open CVN	5
OE6	Cálculo y publicación consistente de indicadores	5
OE7	Visualización de los indicadores en un panel accesible	6
OE8	Benchmark de rendimiento por grado de paralelismo y escala de datos	6
OE9	Robustez operativa mediante uso real y guía de reproducción	6

Tabla 1.1: Relación entre objetivos específicos y capítulos del documento.

1.4. Aspectos éticos y de privacidad en el tratamiento de datos

El tratamiento de datos personales de investigadores exige distinguir entre las dos fuentes empleadas. Los perfiles de ORCID se usan como dato real porque su naturaleza es, por diseño, pública: un identificador ORCID existe precisamente para ser consultado de forma abierta e interoperable entre sistemas de gestión de la investigación, así que su uso aquí no introduce ningún tratamiento distinto del que la propia infraestructura de ORCID habilita.

Los documentos CVN, en cambio, no disfrutan de esa misma naturaleza pública: un currículo completo es información personal habitualmente empleada en solicitudes o evaluaciones privadas, sin un corpus público legítimo del que obtener volumen. Recolectarlos supondría tratar datos personales de terceros sin su consentimiento, un problema de privacidad real para un repositorio público y un trabajo defendido públicamente. Por ello, este documento descarta recolectar CVN reales a volumen y opta por generar currículos sintéticos, válidos frente al esquema oficial y sembrados con campos públicos reales de ORCID, como nombres, obras, filiaciones y fechas, de forma que resulten realistas sin representar a ninguna persona real. Esa misma siembra crea el vínculo por identificador ORCID que la resolución de identidad entre fuentes necesita como clave de fusión.

1.5. Resultados de aprendizaje del máster

El desarrollo de este proyecto se alinea con los resultados de aprendizaje del Máster en Big Data y Computación en la Nube, y no con las competencias de la intensificación de Computación bajo las que se evaluó el TFG anterior. La Tabla 1.2 recoge los seis resultados de aprendizaje sobre los que este trabajo aporta evidencia concreta, junto con los capítulos en los que dicha evidencia se desarrolla.

Resultado	**Descripción**	**Capítulos**
CN02	Arquitecturas de tratamiento masivo, almacenamiento y *pipelines*	2, 3
HA01	Adquisición, fusión y análisis de múltiples fuentes	4, 5
HA02	Procesamiento distribuido y optimización de recursos	5, 6
HA03	Orquestación ETL y almacenamiento escalable	3, 5
CP01	Planificación y despliegue de una solución Big Data	2, 3, 6
CP04	Proyecto profesional original de Big Data	6, 7

Tabla 1.2: Relación entre resultados de aprendizaje del máster y capítulos del documento.

Ninguno de estos resultados se demuestra de forma primaria en el presente capítulo introductorio. Su cumplimiento efectivo se retomará, con evidencia concreta extraída del sistema desarrollado, en el Capítulo 7.

1.6. Estructura del documento

El resto del documento se organiza en seis capítulos adicionales, desde la justificación tecnológica hasta la evaluación y las conclusiones:

- **Capítulo 2: Estado del arte y decisiones arquitectónicas.** Alternativas tecnológicas consideradas para cada componente de la plataforma y justificación de las decisiones adoptadas.
- **Capítulo 3: Infraestructura: despliegue del clúster y servicios base.** Despliegue del clúster Kubernetes local y de los servicios sobre los que se apoya el resto del sistema.

- **Capítulo 4: Ingesta y fusión de fuentes heterogéneas.** Ingesta de datos ORCID reales, generación de CVN sintéticos y su aterrizaje con procedencia en la capa *bronze*.
- **Capítulo 5: Procesamiento distribuido: transformación, resolución de entidades e indicadores.** Validación, normalización y resolución de identidad entre fuentes, y cálculo y publicación de indicadores agregados.
- **Capítulo 6: Visualización, evaluación de rendimiento y endurecimiento.** Panel de indicadores, *benchmark* de rendimiento y defectos de robustez encontrados y corregidos mediante uso real.
- **Capítulo 7: Conclusiones, competencias y trabajo futuro.** Cumplimiento de objetivos, resultados de aprendizaje demostrados, limitaciones y líneas futuras.

2. Estado del arte y decisiones arquitectónicas

El presente capítulo desarrolla el marco tecnológico necesario para justificar la arquitectura de la plataforma construida en este Trabajo Fin de Máster. Se recorre cada capa en el orden en que se despliega: almacenamiento y formato de tabla, sustrato de cómputo, procesamiento distribuido, orquestación de flujos, y consulta y visualización. Para cada una se presentan las alternativas reales del ecosistema y se justifica la opción adoptada frente a ellas. El capítulo cierra con una síntesis que reúne todas las decisiones en una tabla y sirve de puente hacia el Capítulo 3, donde esta arquitectura se despliega efectivamente.

2.1. De los sistemas de almacenamiento analítico al lakehouse

Un sistema que integra currículos CVN y perfiles de investigador ORCID necesita almacenar datos con dos exigencias que tradicionalmente se han resuelto con arquitecturas distintas: la escala y variedad propias de un *data lake*, y las garantías transaccionales de un *data warehouse*.

Un almacén de datos clásico, o *data warehouse*, define el esquema antes de cargar los datos, una disciplina conocida como *schema-on-write*. Esto aporta consistencia y consultas analíticas fiables, justo lo que necesita este proyecto para publicar indicadores. Su inconveniente aparece antes: ingerir una fuente nueva exige transformarla primero para que encaje en ese esquema, un proceso costoso cuando CVN y ORCID llegan en formatos semiestructurados que no coinciden entre sí.

Un lago de datos clásico, o *data lake*, resuelve el problema contrario. Almacena cualquier fuente tal cual llega y solo interpreta su esquema al leerla, una práctica conocida como *schema-on-read*. Sin metadatos transaccionales, sin embargo, corre el riesgo documentado

del *data swamp*: lecturas inconsistentes durante escrituras concurrentes, y ninguna forma de auditar cómo estaban los datos en un momento anterior.

El *lakehouse* combina ambas exigencias añadiendo una capa de metadatos transaccionales, un formato de tabla abierto, sobre almacenamiento de objetos barato. Esa capa aporta transacciones ACID, evolución de esquema controlada y *time travel*, sin mover los datos a un motor relacional cerrado. Armbrust et al. [Armbrust et al., 2021] formulan esta convergencia como respuesta directa a las limitaciones de los dos modelos anteriores. La presentan como la arquitectura de referencia cuando conviven fuentes documentales y de volumen masivo, exactamente la situación de este proyecto.

La arquitectura adoptada es, por tanto, un *lakehouse* organizado en tres capas: *bronze*, *silver* y *gold*. La capa *bronze* conserva la procedencia de cada fuente. La capa *silver* aplica resolución de identidad y control de calidad. La capa *gold* deja agregados listos para consumo. Este es el patrón de fusión multi-fuente que exige el problema planteado en el Capítulo 1. Mantener un único sistema para ambas exigencias, en lugar de un lago para ingesta y un almacén separado para publicación, evita además duplicar la superficie operativa del proyecto.

2.2. Formato de tabla: Apache Iceberg frente a sus alternativas

El formato de tabla es la pieza que hace posible un *lakehouse*: una capa de metadatos sobre ficheros en almacenamiento de objetos que los convierte en algo que se comporta como una tabla, con transacciones y evolución de esquema. Tres formatos tienen adopción real en el ecosistema actual.

Apache Iceberg [Apache Software Foundation, 2026f] nació en Netflix para superar los límites de las tablas Hive a escala de petabytes, y hoy es un proyecto de la Apache Software Foundation con gobierno multi-vendedor. Aporta evolución de esquema y de partición sin reescribir los datos existentes, y aislamiento de *snapshots* que permite leer una tabla de forma consistente mientras otro proceso escribe en ella. Cuenta además con soporte amplio en varios motores de consulta, entre ellos Spark, Trino y Flink, lo que significa que el mismo catálogo podría consultarse en el futuro desde un motor distinto sin cambiar el formato de tabla.

Delta Lake [Delta Lake Project, Linux Foundation, 2026] nació en Databricks y fue donado a la Linux Foundation. Su integración con el ecosistema Spark de Databricks es muy pulida, pero sus funciones más avanzadas alcanzan su rendimiento completo sobre el *runtime* propietario de Databricks. Fuera de ese entorno, su soporte multi-motor ha sido históricamente más limitado. Apache Hudi [Apache Software Foundation, 2026e] nació en Uber y optimiza actualizaciones incrementales de baja latencia sobre flujos continuos, mediante

upserts y captura de cambios de datos, o CDC. Esta ventaja no aplica a este proyecto: es un *pipeline* por lotes que reconstruye *silver* y *gold* por completo en cada ejecución.

La Tabla 2.1 resume las tres opciones.

Formato	Origen y gobierno	Ajuste con este proyecto
Apache Iceberg	Netflix, hoy proyecto Apache, gobierno multi-vendedor	Sin dependencia de un proveedor, consultable desde otros motores sin cambiar el formato de tabla
Delta Lake	Databricks, donado a la Linux Foundation	Funciones avanzadas afinadas para el *runtime* de Databricks, menor soporte multi-motor fuera de él
Apache Hudi	Uber, orientado a actualizaciones incrementales de baja latencia	Optimizado para CDC y *streaming*, este proyecto es un *pipeline* por lotes

Tabla 2.1: Comparación de formatos de tabla abierta para arquitecturas *lakehouse*.

La opción adoptada es Apache Iceberg. Al no depender de un *runtime* comercial, las tablas creadas aquí seguirían siendo consultables sin cambios si, como trabajo futuro, la plataforma migrase a un motor de consulta interactivo como Trino, tratado en la Sección 2.7, o a un clúster gestionado en la nube. Delta Lake queda como alternativa razonable, pero condicionada a un entorno Databricks que este proyecto no usa. Hudi se descarta por no encajar con un *pipeline* por lotes.

2.2.1. Catálogo de Iceberg

Iceberg necesita además un catálogo que resuelva el nombre de una tabla a la ubicación de su metadato raíz. Las opciones habituales son:

- **Hive Metastore [Apache Software Foundation, 2026d]**: servicio centralizado que almacena la estructura de tablas y particiones. Exige desplegar y operar un servicio adicional únicamente para esa resolución de nombres.
- **Catálogo REST [Apache Software Foundation, 2026f]**: la vía que el propio proyecto Iceberg promueve como estándar de interoperabilidad entre motores. Resuelve el mismo problema mediante una API HTTP común, pero exige igualmente un servicio adicional.

- **Project Nessie [Project Nessie, 2026]**: catálogo versionado al estilo Git, con ramas y *commits* sobre el catálogo completo. Muy atractivo para múltiples escritores que necesitan aislar sus cambios, pero comparte la misma exigencia operativa.
- **Catálogo Hadoop**, basado en rutas: resuelve el nombre de tabla directamente contra una convención de directorios en el propio almacenamiento de objetos, sin ningún servicio adicional.

La opción adoptada es el catálogo Hadoop. Con un único escritor por capa y sin necesidad de versionar el catálogo a nivel de repositorio, el coste operativo de un servicio adicional no se compensa con ninguna capacidad que el proyecto vaya a usar. El versionado de Nessie y la interoperabilidad de un catálogo REST quedan como trabajo futuro. Ningún escenario del proyecto las necesita todavía, no por ninguna carencia técnica de esas dos opciones.

2.3. Almacenamiento de objetos: MinIO frente a sus alternativas

HDFS [Apache Software Foundation, 2026c], la opción clásica de Big Data, acopla almacenamiento y cómputo en los mismos nodos, lo que contradice el propio patrón *lakehouse* y añade una pieza operativa adicional sin aportar ventaja alguna aquí. El almacenamiento nativo de un proveedor de nube pública es la opción de producción real, pero es incompatible con un despliegue autoalojado y de coste cero. MinIO [MinIO, Inc., 2026], un servidor de objetos autoalojado compatible con la API de Amazon S3, resuelve ambos problemas a la vez: el mismo protocolo, `s3a://`, que usarían los trabajos de procesamiento contra un proveedor real funciona sin cambios contra MinIO.

La opción adoptada es MinIO, precisamente por esa compatibilidad de protocolo. El código de acceso a almacenamiento escrito contra MinIO no necesitaría cambios para apuntar a un proveedor de nube real.

2.4. Sustrato de orquestación de contenedores y entorno de despliegue

Los servicios de la plataforma pueden desplegarse de tres formas:

- **Sin orquestador**: cada servicio como proceso o contenedor suelto en la máquina. Es la opción más simple de depurar, pero renuncia al modelo declarativo de infraestructura versionada y no ejercita ninguna competencia de computación en la nube.

- **Docker Swarm [Docker, Inc., 2026]**: orquestador más simple que Kubernetes, pero prácticamente ausente del soporte oficial del ecosistema Big Data. Ni Spark, ni Airflow, ni Superset publican un camino de despliegue nativo equivalente al que sí publican para Kubernetes.
- **Kubernetes [Cloud Native Computing Foundation, 2026]**: el estándar de facto, con soporte de primera clase en el resto de la pila elegida.

La opción adoptada es Kubernetes, y dentro de él, k3s [K3s Project Authors, 2026]: una distribución certificada, empaquetada como binario único con huella de recursos reducida, apta para una sola máquina de desarrollo sin dejar de ser Kubernetes real. Esa condición hace posible desplegar sobre una única máquina local durante el desarrollo, en lugar de sobre un clúster real en la nube. Evita así el coste de mantener infraestructura en la nube encendida mientras el sistema todavía cambia con frecuencia.

La arquitectura no depende de esa máquina concreta, porque todos los servicios se despliegan con *charts* de Helm sobre Kubernetes estándar. La única excepción es el modo de red `host-gw`, necesario bajo WSL2 para evitar un problema conocido de encapsulado UDP en el *backend* por defecto, una decisión de compatibilidad con el entorno de desarrollo, no una limitación de Kubernetes. El mismo despliegue es, por tanto, replicable sobre un clúster real en la nube cambiando la infraestructura subyacente, no la definición de los servicios. Ese paso queda documentado como trabajo futuro.

2.5. Procesamiento distribuido: Apache Spark frente a sus alternativas

El procesamiento de este proyecto tiene forma de lotes programados, no de flujo continuo de eventos, lo que orienta la elección entre los motores disponibles:

- **Apache Flink [Apache Software Foundation, 2026b]**: pensado ante todo para baja latencia sobre flujos continuos. Añadiría la complejidad operativa de su propia topología de coordinación sin que el proyecto llegue a explotar esa ventaja.
- **Dask [Dask Development Team, 2026] y Ray [Anyscale, Inc., 2026]**: motores nativos de Python, con curva de adopción más suave. Su soporte de Iceberg es bastante menos maduro, y su presencia en arquitecturas *lakehouse* de referencia es minoritaria.
- **Apache Spark [Zaharia et al., 2016]**: descrito por sus autores como un motor unificado para lotes, flujos, SQL y aprendizaje automático. Cuenta con la integración

más madura del ecosistema con Iceberg y con un *backend* nativo sobre Kubernetes [Apache Software Foundation, 2026h].

La opción adoptada es Apache Spark, lanzado con `spark-submit` en modo Kubernetes *client* desde el propio orquestador, en lugar de mediante el *Spark Kubernetes Operator* [Kubeflow, 2026]. Este último exige instalar y depurar un controlador adicional, mientras que `spark-submit` en modo *client* sigue programando los ejecutores como pods reales sin esa pieza extra. Ciertas propiedades de configuración del controlador de Spark resultaron no operativas en modo *client* y tuvieron que trasladarse a la especificación del propio pod, un ajuste documentado en el Capítulo 3.

2.6. Orquestación de flujos ETL: Apache Airflow frente a sus alternativas

Cuatro herramientas cubren la orquestación de flujos ETL en el ecosistema actual:

- **Prefect [Prefect Technologies, Inc., 2026]**: ergonomía de desarrollo local moderna, con los flujos definidos como código Python idiomático, pero una madurez de despliegue nativo en Kubernetes menor que la de Airflow.
- **Dagster [Dagster Labs, 2026]**: modelo de activos de datos con linaje explícito entre pasos, atractivo para un lago de datos, pero no crítico para un *pipeline* de solo tres etapas cuya procedencia ya resuelve el propio formato Iceberg.
- **Luigi [Spotify, 2026]**: simplicidad orientada a tareas con dependencias, pero sin interfaz de administración ni un ejecutor de Kubernetes nativo comparable.
- **Apache Airflow [Apache Software Foundation, 2026a]**: el orquestador incumbente, con *chart* de Helm oficial y un ejecutor de Kubernetes de primera clase, el `KubernetesExecutor`, en el que cada tarea se ejecuta como su propio pod.

La opción adoptada es Apache Airflow con su ejecutor de Kubernetes. Frente a su alternativa madura, el `CeleryExecutor`, basado en un intermediario de colas, reutiliza el propio clúster como sustrato de ejecución en lugar de exigir levantar una cola de mensajes y un almacén de resultados adicionales.

2.7. Capa de consulta interactiva y capa de visualización

Trino [Trino Software Foundation, 2026], un motor de consulta SQL federado de baja latencia sobre tablas Iceberg, sería útil si el proyecto necesitase una vía de consulta independiente del panel de visualización. No es el caso, porque todo el acceso a los datos ya se hace desde los propios trabajos de procesamiento, así que queda como trabajo futuro.

Para la visualización, cuatro opciones son relevantes:

- **Grafana [Grafana Labs, 2026]**: la referencia para paneles de observabilidad y series temporales, pero encaja peor con gráficos analíticos agregados sobre un esquema relacional, la forma real de los indicadores de este proyecto.
- **Power BI [Microsoft, 2026] y Tableau [Salesforce, Inc., 2026]**: las herramientas de BI propietarias de referencia del mercado, incompatibles con la naturaleza autoalojada y de coste cero de la plataforma.
- **Metabase [Metabase, Inc., 2026]**: alternativa de código abierto comparable a Superset, de entrada más simple, pero con un modelo de panel menos expresivo para versionar el panel completo como código.
- **Apache Superset [Apache Software Foundation, 2026g]**: permite exportar y reconstruir el panel completo desde el repositorio, en lugar de depender de un artefacto manual imposible de versionar.

La opción adoptada es Apache Superset, conectado a una base de datos relacional y no a Iceberg. Con Trino ya descartado, materializar las tablas de *gold*, ya agregadas y pequeñas, en una base de datos relacional como último paso del *pipeline* da a Superset un conector nativo sin necesitar ningún servicio intermedio. El *chart* de Helm oficial de Superset resultó estar obsoleto frente a la vía de despliegue que el proyecto promueve actualmente. Se gestionó fijando su versión y construyendo una imagen propia, en lugar de adoptar un operador todavía en maduración, una decisión que se retoma en el Capítulo 6.

2.8. Síntesis y arquitectura resultante

La Tabla 2.2 reúne, para cada capa, la decisión adoptada, la alternativa principal descartada y el motor de su justificación.

Decisión	**Alternativa principal descartada**	**Motor de la justificación**
Arquitectura *lakehouse*	Almacén y lago separados	Un único sistema para ambas exigencias, sin duplicar operación
Apache Iceberg	Delta Lake, Apache Hudi	Sin dependencia de proveedor, consultable desde otros motores sin cambios
Catálogo Hadoop	Hive Metastore, REST, Nessie	Un único escritor por capa, sin necesidad de versionado de catálogo
MinIO	HDFS, nube pública real	Mismo protocolo que un proveedor real, sin coste ni dependencia externa
Kubernetes k3s, nodo local	Docker Swarm, clúster real en la nube	Soporte de primera clase en el resto de la pila, coste y reproducibilidad durante el desarrollo
Apache Spark, `spark-submit` sin operador	Apache Flink, Dask, Ray, *Spark Kubernetes Operator*	Patrón por lotes del proyecto, integración madura con Iceberg, menor superficie operativa
Apache Airflow, ejecutor de Kubernetes	Prefect, Dagster, Luigi, `CeleryExecutor`	Despliegue Kubernetes-nativo maduro, sin servicios adicionales
Superset sobre base de datos relacional, sin Trino	Trino, Grafana, Metabase, BI propietario	Panel como código, sin servicio intermedio adicional

Tabla 2.2: Síntesis de las decisiones arquitectónicas de la plataforma.

En conjunto, estas decisiones configuran una arquitectura *lakehouse* coherente, autoalojada y reproducible desde cero, con soporte de primera clase entre todos sus componentes y con cada alternativa descartada documentada como trabajo futuro explícito. El Capítulo 3 describe cómo se despliega efectivamente. Los Capítulos 4, 5 y 6 retoman, cada uno en su

lugar, las decisiones de estrategia de datos, resolución de identidad y observabilidad propias del desarrollo de cada capa.

3. Infraestructura: despliegue del clúster y servicios base

El presente capítulo describe cómo se construye, en la práctica, la base de infraestructura sobre la que corre el resto del sistema. Se recorren el clúster Kubernetes local, los servicios centrales que se despliegan sobre él, el catálogo Iceberg configurado sobre el almacenamiento de objetos y la ejecución de Spark orquestada por Airflow. Esta última demuestra, de extremo a extremo, que el mecanismo central de la plataforma funciona antes de construir ningún flujo de negocio sobre él. El Capítulo 2 justifica cada una de estas decisiones frente a sus alternativas. Este capítulo describe cómo se llevaron a la práctica, con los hallazgos operativos reales que surgieron al hacerlo.

3.1. Arquitectura de despliegue

La plataforma se organiza en una cadena de capas. Un orquestador de flujos, Airflow, dispara trabajos de cómputo distribuido con Spark sobre Kubernetes. Estos trabajos leen y escriben contra un almacenamiento de objetos con formato de tabla, MinIO con Iceberg. Los resultados agregados se publican en una capa relacional, PostgreSQL. Finalmente, esos resultados se consumen desde una capa de visualización, Superset, que se describe en el Capítulo 6. Todos estos servicios se despliegan como *charts* de Helm [The Helm Authors, 2026]. La Figura 3.1 resume esta cadena.

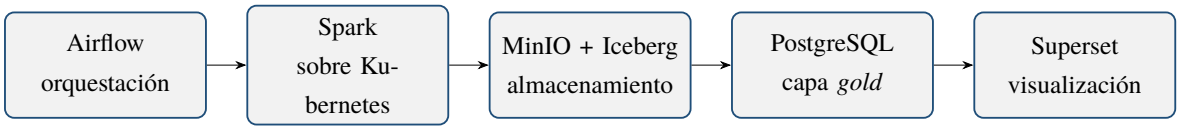


Figura 3.1: Cadena de capas de la plataforma.

El clúster subyacente es Kubernetes [Cloud Native Computing Foundation, 2026]. Corre en un único nodo local, con un espacio de nombres dedicado llamado `tfm-lakehouse`. La Tabla 3.1 resume los servicios desplegados y su papel en esta cadena.

Servicio	**Tecnología**	**Versión**	**Rol**
Clúster	k3s	v1.36.4+k3s1	Sustrato Kubernetes de un único nodo
Almacenamiento	MinIO	*chart* 17.0.21	Objetos S3, cubo `lakehouse`
Base relacional	PostgreSQL	*chart* 18.11.3	Capa *gold* publicada
Orquestación	Airflow	*chart* 1.22.0	Flujos ETL, ejecutor de Kubernetes
Catálogo de tablas	Iceberg	1.11.0, *runtime* Spark 3.5	Catálogo Hadoop sobre MinIO
Procesamiento	Spark	3.5.9	Trabajos distribuidos lanzados desde Airflow

Tabla 3.1: Servicios desplegados en el clúster local.

3.2. Clúster Kubernetes local

El clúster se instaló como distribución k3s, en modo de un único nodo. Se configuró con el modo de red `host-gw` en lugar del *backend* VXLAN por defecto, que evita un problema conocido de encapsulado UDP bajo el entorno de desarrollo WSL2 empleado. k3s se instala como servicio `systemd`, lo que exige que dicho gestor de servicios esté habilitado en el entorno. Bajo WSL2 esto no viene activado por defecto y requiere un paso de configuración explícito antes de instalar el clúster.

Sobre el nodo así preparado se creó el espacio de nombres dedicado `tfm-lakehouse`. También se añadieron los repositorios oficiales de Helm de Apache Airflow y Apache Super-

set. Los *charts* de MinIO y PostgreSQL, en cambio, se consumen por referencia OCI directa en lugar de mediante un repositorio Helm clásico, tal y como se detalla en la Sección 3.3.

3.3. Servicios base: MinIO, PostgreSQL y Airflow

Los tres servicios centrales se despliegan mediante Helm sobre el clúster. MinIO se configura con un único cubo llamado `lakehouse`, con un único prefijo de aterrizaje de ficheros crudos, `bronze`. La Sección 3.4 explica por qué el cubo no necesita ningún otro prefijo. PostgreSQL [The PostgreSQL Global Development Group, 2026] se despliega como una instancia dedicada a la futura capa *gold*. Esta instancia se mantiene deliberadamente separada de la base de datos de metadatos que el propio *chart* de Airflow ya incorpora de forma integrada, para no acoplar dos sistemas con ciclos de vida distintos. Airflow se despliega con el ejecutor de Kubernetes, `KubernetesExecutor`, y con persistencia de sus DAG en un volumen propio. Los ficheros de flujo se entregan copiándolos directamente a dicho volumen, en lugar de mediante sincronización continua con un repositorio Git.

Durante el despliegue se constató un hallazgo operativo real, no anticipado en el planteamiento inicial. El catálogo gratuito de Bitnami, proveedor de los *charts* de MinIO y PostgreSQL empleados, redujo drásticamente su oferta de imágenes públicas a lo largo de 2025. Todas las imágenes anteriores al recorte se movieron a un registro congelado, `bitnamilegacy`, que sigue siendo público y sin autenticación pero que no recibe ya ninguna actualización de seguridad [Broadcom Inc. / Bitnami Charts Contributors, 2025]. Este hallazgo obligó a fijar explícitamente, para cada imagen de MinIO y PostgreSQL, tanto el registro como la etiqueta exacta. Cada una se verificó como descargable en el momento del despliegue, en lugar de asumirse de la documentación de cada *chart*. Se acepta como una limitación conocida, razonable para un clúster local sin exposición a Internet, pero que exigiría revisarse antes de cualquier despliegue real.

3.4. Catálogo Iceberg sobre MinIO

El catálogo Hadoop de Iceberg se configura mediante un único catálogo de Spark, llamado `lakehouse`, con su raíz de almacén en `s3a://lakehouse/warehouse`. Esta ruta se mantiene deliberadamente distinta del prefijo `bronze` que aloja los ficheros crudos aterrizados directamente. Mezclar ambos bajo el mismo prefijo arriesgaría que la lógica interna de listado y *commit* de Iceberg tropezase con objetos ajenos a sus propias tablas. Dentro de ese almacén, las capas *bronze*, *silver* y *gold* se representan como espacios de nombres de Iceberg,

es decir `lakehouse.bronze`, `lakehouse.silver` y `lakehouse.gold`, no como catálogos independientes. Esto evita triplicar la superficie de configuración sin necesidad funcional alguna. La Figura 3.2 muestra esta separación dentro del mismo cubo.

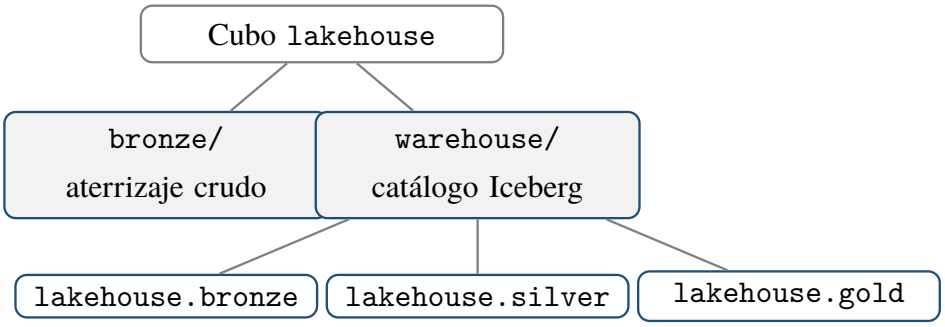


Figura 3.2: Separación, dentro del mismo cubo, entre el prefijo de aterrizaje crudo y las tablas Iceberg del catálogo.

El cubo se aprovisionó al principio con tres prefijos de aterrizaje, `bronze`, `silver` y `gold`, a semejanza de esos mismos tres espacios de nombres. La ingesta del Capítulo 4 aterriza en `bronze` los ficheros crudos de ORCID y los CVN sintéticos, porque son datos que entran a la plataforma desde fuera y necesitan un primer punto de aterrizaje antes de cualquier procesamiento. Las capas *silver* y *gold*, en cambio, no llegan a aterrizar ningún fichero crudo propio. Se calculan enteramente dentro de la plataforma, mediante trabajos de Spark que leen *bronze* y escriben su resultado directamente como tablas Iceberg bajo el espacio de nombres correspondiente de `warehouse/`. Al quedar claro esto durante el diseño de la ingesta y la transformación, los prefijos `silver` y `gold` del cubo se retiraron del diseño por no cumplir ninguna función, y el cubo quedó con el único prefijo de aterrizaje que sí se usa, `bronze`.

La configuración de acceso exigió, además, fijar una cadena de versiones mutuamente compatibles entre Spark, el *runtime* de Iceberg para Spark y las bibliotecas de acceso a S3 de Hadoop. Esta configuración incluye el punto de conexión S3, el direccionamiento por ruta en lugar de por subdominio, y las credenciales inyectadas por variable de entorno desde el mismo secreto de Kubernetes que usa MinIO. Una combinación incorrecta de las versiones de Spark, Iceberg y Hadoop es una fuente bien documentada de errores de *classpath* difíciles de diagnosticar. Esta cadena de versiones se verificó de dos formas antes de darla por buena. Primero, que cada artefacto se pudiera descargar realmente desde su origen oficial. Después, extrayendo la propia distribución de Spark para comprobar qué versión de los clientes de Hadoop incluía en la práctica, en lugar de asumirla de la documentación.

3.5. Ejecución de Spark desde Airflow

Cada trabajo de Spark se lanza desde una tarea de Airflow que usa `KubernetesPodOperator` y ejecuta `spark-submit` en modo Kubernetes *client*, contra una imagen de Spark propia que ya incorpora las dependencias de Iceberg y S3A verificadas en la Sección 3.4. El propio pod de la tarea de Airflow actúa así como el proceso controlador de Spark, llamado *driver*. Este proceso crea y destruye sus propios pods ejecutores en el mismo espacio de nombres. Para ello se creó una cuenta de servicio dedicada de Kubernetes con permisos explícitos, limitados a ese espacio de nombres, en lugar de un permiso de alcance global.

La verificación de este mecanismo, antes de construir ningún flujo de negocio sobre él, se hizo con un trabajo mínimo. Este trabajo crea un espacio de nombres y una tabla Iceberg de prueba, inserta unas pocas filas y las relee. Este primer intento reveló varios problemas reales que no eran evidentes desde el diseño:

- **Credenciales del controlador.** La configuración de Spark para inyectar credenciales en el controlador y en los ejecutores solo tiene efecto real sobre los ejecutores en modo *client*. En ese modo, el controlador es el propio pod que ya lanzó `spark-submit`, así que esas propiedades se ignoran de forma silenciosa sobre él. Las credenciales del controlador se resolvieron fijándolas directamente sobre la especificación del propio pod de Airflow, no mediante configuración de Spark.
- **Punto de entrada de la imagen.** Sustituir por completo el punto de entrada de la imagen oficial de Spark para ejecutar `spark-submit` directamente evita un paso interno de dicha imagen que adapta el sistema de usuarios del contenedor al identificador de usuario arbitrario con el que se ejecuta el pod. Omitir ese paso produce errores de bajo nivel en la resolución del directorio de usuario y en el inicio de sesión de Hadoop, que no delatan su causa real. La solución consiste en invocar `spark-submit` como argumento del punto de entrada original, no en sustituirlo.
- **Permisos de limpieza.** Los permisos concedidos a la cuenta de servicio cubrían la creación y el borrado individual de pods, pero no el borrado por lotes que Spark efectúa al finalizar un trabajo para limpiar sus propios ejecutores. Sin ese permiso adicional, el trabajo en sí terminaba correctamente, pero el pod controlador quedaba en estado de error por el fallo de limpieza posterior.

3.6. Verificación de extremo a extremo

Una vez corregidos los tres puntos anteriores, la prueba se completó con dos verificaciones independientes. Este patrón de comprobación se repite en los capítulos siguientes cada vez que se demuestra un resultado nuevo. En primer lugar, el propio trabajo debía terminar con éxito y su registro debía confirmar el número de filas esperado. En segundo lugar, un pod completamente distinto, sin relación con el trabajo que escribió los datos, debía poder leer la misma tabla a través del mismo catálogo y obtener el mismo resultado. Ambas verificaciones se cumplieron. Esto confirma que el mecanismo central de almacenamiento y procesamiento distribuido de la plataforma funciona de extremo a extremo, antes de construir sobre él ningún flujo de ingesta o transformación real.

Esta forma de verificar, y no solo la corrección puntual de cada fallo, sostiene el objetivo de reproducibilidad de todo el sistema. La plataforma se diseñó para poder reconstruirse desde cero siguiendo una guía escrita, y esa guía se valida en la práctica, no solo se redacta, como se retoma en el Capítulo 6.

4. Ingesta y fusión de fuentes heterogéneas

El presente capítulo describe cómo se adquieren, generan y aterrizan las dos fuentes de datos del sistema. Se recorren la doble adquisición de datos de ORCID, la generación de currículos CVN sintéticos sembrados con esos mismos datos, el aterrizaje en bronze con procedencia y la verificación de extremo a extremo del flujo de Airflow que encadena las cuatro tareas. El enlace deliberado entre el CVN sintético y su semilla ORCID, introducido en este capítulo, es la clave de unión que la resolución de identidad del Capítulo 5 explota.

4.1. Arquitectura de la ingesta

La ingesta se implementa como un único flujo de Airflow, `ingest_validate`, con cuatro tareas encadenadas: obtención del subconjunto ORCID masivo, generación de CVN sintético, enriquecimiento vía la API de ORCID y validación con aterrizaje en bronze. Este orden no es arbitrario. La tarea de enriquecimiento consulta la API por los identificadores ORCID que los CVN sintéticos ya declaran, por lo que no puede ejecutarse antes de que existan. Ese acoplamiento deliberado entre tareas es la propia demostración de fusión dirigida entre fuentes: el identificador que une un CVN sintético con su semilla real es el mismo que la tarea de enriquecimiento usa para pedir datos frescos a la API. La Figura 4.1 resume el orden de las cuatro tareas.

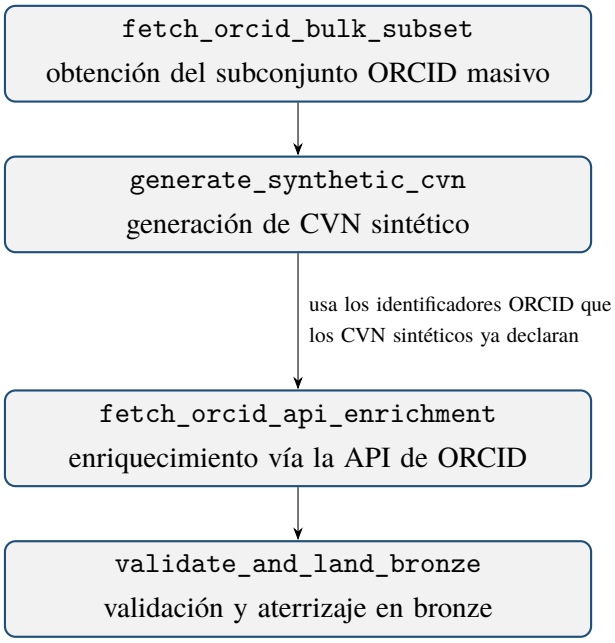


Figura 4.1: Orden de tareas del flujo `ingest_validate`.

Cada tarea corre en un pod independiente lanzado con `KubernetesPodOperator`, sobre una imagen propia con Python 3.14 y las dependencias del proyecto. El propio Airflow corre sobre Python 3.13, así que la lógica de negocio del proyecto no puede ejecutarse en sus *workers* y necesita, igual que la ejecución de Spark del Capítulo 3, un pod dedicado por tarea. El código del proyecto, los esquemas y los datos llegan a cada pod montados directamente desde el disco del nodo, sin necesidad de reconstruir la imagen en cada cambio de código. Las tareas se comunican entre sí a través de ese mismo directorio de datos compartido, con un subdirectorio por ejecución. El mecanismo nativo de paso de mensajes de Airflow se reserva para resúmenes pequeños, como recuentos y rutas, y nunca para los propios datos, que alcanzan varios gigabytes por ejecución.

4.2. Adquisición de datos de ORCID

ORCID [ORCID, 2026b] ofrece dos mecanismos públicos y oficiales para obtener datos, y el sistema usa ambos porque cubren necesidades distintas y complementarias. Ninguno de los dos sustituye al otro.

- **API pública de ORCID [ORCID, 2026a].** Resuelve consultas puntuales por identificador y da acceso al registro más reciente de una persona concreta. Su límite de tasa,

en el nivel anónimo sin autenticación, es de 25 000 lecturas al día y 12 peticiones por segundo por dirección IP. Durante la implementación se encontró una discrepancia entre la documentación pública sobre el registro de aplicaciones cliente, que describe el nivel autenticado como accesible a cualquier investigador individual, y el formulario de registro real, que exige describir la aplicación como herramienta de una organización miembro. Ese requisito no encaja con un proyecto académico personal, de modo que el sistema usa el nivel anónimo, sin que ello afecte a la demostración de fusión dirigida entre fuentes.

- **Fichero público de datos de ORCID.** ORCID publica anualmente un volcado completo de sus perfiles públicos en la plataforma de datos abiertos Figshare [Figshare, 2026]. Este fichero da volumen real en lugar de tener que sintetizar también los datos de ORCID. El fichero completo se distribuye en doce archivos, de los cuales solo se usa el de resúmenes de perfil, de unos 46 GB, y no los once de actividades detalladas, de unos 191 GB en conjunto, que exceden lo que este trabajo necesita. El sistema recorre el fichero de resúmenes completo, sin detenerse al alcanzar un volumen objetivo, para no introducir un sesgo de muestreo hacia los primeros registros del archivo, y filtra los perfiles con afiliación declarada en España [ORCID, 2025]. La integridad del fichero descargado se verifica contra la suma de comprobación que la propia Figshare publica junto a él.

La Tabla 4.1 resume los dos mecanismos.

Mecanismo	Uso	Volumen	Límite
API pública	Enriquecimiento dirigido por identificador	Un registro por consulta	25 000 lecturas/día, 12 peticiones/s
Fichero de datos	Volumen real de siembra para la generación sintética	301 763 perfiles con afiliación española filtrados	Instantánea anual, sin límite de tasa

Tabla 4.1: Los dos mecanismos públicos de adquisición de datos de ORCID.

El filtrado del fichero completo, sobre 26 078 951 registros escaneados, dejó 301 763 perfiles con afiliación española, un 1,157% del total. La primera implementación, que procesaba

el archivo en flujo directamente desde la red, proyectaba unas siete horas de ejecución. El cuello de botella no estaba en la red sino en el análisis de XML de cada registro, así que la solución fue descargar antes el archivo al disco local y aplicar un filtro previo a nivel de bytes que descarta un registro sin analizarlo completamente cuando resulta evidente que no puede coincidir. Con ese cambio, la extracción completa tardó 81,6 minutos. El resultado se guarda como un fichero XML por perfil, organizado en subdirectorios por prefijo del identificador, que sirve de semillero a la generación de CVN sintéticos descrita a continuación.

4.3. Generación de currículos CVN sintéticos

El sistema necesita un volumen realista de currículos CVN para demostrar la plataforma a escala, y ese volumen no puede provenir de currículos reales por las razones de privacidad expuestas en el Capítulo 1. La solución adoptada genera documentos Open CVN sintéticos, válidos contra el mismo esquema JSON Schema [JSON Schema Organization, 2026] que define el formato oficial del proyecto, y los siembra con campos públicos reales tomados del subconjunto ORCID descrito en la Sección 4.2: nombre, afiliaciones, títulos de obras y palabras clave. Nunca se copian de ORCID la biografía, el correo electrónico ni las URL de investigador de un perfil, porque ORCID pide explícitamente no presentar datos modificados como genuinos, aunque su fichero de datos se publique bajo licencia CC0. El resto de la identidad de cada documento, como el teléfono o la fecha del documento, se inventa por completo, con valores que no pueden confundirse con datos reales, y cada documento generado queda marcado como sintético en sus propios metadatos.

Una parte configurable de los documentos generados, un 70% en la configuración de referencia, declara el identificador ORCID de su semilla en el campo del esquema reservado a identificadores digitales de autor, con el código de fuente que las tablas de referencia del CVN asignan a ORCID. El resto conserva el nombre y las afiliaciones de su semilla, con variaciones realistas, pero sin ese identificador. Un fichero de control aparte, no aterrizado en el almacenamiento de objetos, registra para cada documento generado el identificador de su semilla y el tipo de enlace aplicado. Esa combinación de documentos con enlace declarado, documentos con coincidencia solo por nombre y afiliación, y documentos sin relación entre sí, es precisamente la variedad de casos que la resolución de identidad del Capítulo 5 necesita para verificarse, y el fichero de control es su verdad de referencia. Cuando el volumen solicitado supera el número de semillas disponibles, una semilla se reutiliza para varios documentos, con una selección distinta de sus obras y una identidad inventada distinta en cada uno, pero sin alterar fechas, organizaciones ni otros hechos tomados de ORCID.

La validación de cada documento generado ocurre en dos capas. La primera reutiliza el validador Open CVN del TFG [Jaén, 2026] sobre el documento completo. La segunda, propia de este generador, valida además cada entrada contra la definición estricta de su propio tipo de entidad dentro del esquema. Esta segunda capa resultó necesaria en la práctica y no solo en el diseño. El esquema del documento completo deja el contenido de cada entrada sin restricciones de campos, por lo que un documento con campos inventados pasaría la primera capa sin problema. La primera ejecución de prueba generó 200 documentos con solo dos errores de construcción, y aun así 179 de ellos resultaron inválidos frente a la segunda capa, por dos campos con nombre incorrecto y una fecha vacía en un campo obligatorio. Corregidos ambos errores, las ejecuciones posteriores no descartaron ningún documento. Una ejecución de 10 000 documentos, con semilla fija y el mismo reparto de enlace del 70%, generó 7 000 documentos enlazados y 3 000 sin enlazar, sin ningún documento inválido, en 169 segundos.

4.4. Aterrizaje en bronze con procedencia

Las tres fuentes de datos, el subconjunto ORCID masivo, el enriquecimiento por API y los CVN sintéticos, aterrizan como objetos de texto plano en formato JSON Lines bajo el prefijo `bronze` del cubo de MinIO configurado en el Capítulo 3, organizados por fuente y por fecha de ingesta. No se escriben como tablas Iceberg en esta fase: mantener el aterrizaje crudo separado del catálogo de tablas evita que la lógica interna de Iceberg tropiece con objetos ajenos a sus propias tablas, y deja la creación de las tablas bronze de Iceberg como parte del trabajo de transformación del Capítulo 5.

Cada registro aterrizado se envuelve en un sobre de procedencia común a las tres fuentes, resumido en la Tabla 4.2. El propio contenido original, ya sea el XML de ORCID sin modificar, el JSON que devuelve la API o el documento CVN sintético, viaja como carga útil dentro de ese sobre. Este vocabulario de procedencia extiende el mismo utilizado por el bloque de trazabilidad del formato Open CVN del TFG, en lugar de inventar una convención nueva.

Campo	Contenido
Identificador de registro	Identifica el registro de forma única dentro de su fuente
Fuente de origen	Cuál de las tres fuentes produjo el registro
Referencia al origen	Referencia concreta dentro de esa fuente, por ejemplo un identificador ORCID
Momento de obtención	Cuándo se obtuvo el dato de su fuente original
Momento de aterrizaje	Cuándo se aterrizó el registro en bronze
Identificador de ejecución	Qué ejecución del flujo aterrizó el registro
Resultado de la comprobación	Estado de la validación descrita a continuación

Tabla 4.2: Campos del sobre de procedencia, comunes a las tres fuentes.

Antes de aterrizar, cada registro pasa una comprobación de calidad ligera, deliberadamente más simple que la resolución de identidad del Capítulo 5. Para ORCID exige un XML bien formado, un identificador con dígito de control válido y al menos un nombre. Para el CVN sintético exige que el documento pase el validador Open CVN del TFG. Los registros que no superan esta comprobación se separan a una zona de rechazados, con sus errores adjuntos, en lugar de descartarse sin dejar rastro, y la propia tarea de aterrizaje falla si la proporción de rechazados de una ejecución supera un umbral del 5%, para convertir un fallo sistemático de origen en un error visible en vez de en un aterrizaje silenciosamente incompleto. Sobre los primeros 20 000 registros ORCID reales, la tasa de rechazo medida fue del 2,66%, y su causa casi exclusiva fue la ausencia de un apellido público en el perfil.

Aterrizar el fichero masivo completo cada día no tiene sentido: su contenido no cambia entre ejecuciones y ocupa varias decenas de gigabytes. Por ello, el subconjunto masivo solo se aterriza una vez por instantánea del fichero de origen, con un límite de 20 000 registros por defecto que mantiene cada ejecución diaria en un tamaño manejable. Ese límite es un parámetro de la propia ejecución, así que puede elevarse para obtener un volumen mayor cuando haga falta, como en el trabajo de rendimiento del Capítulo 6.

4.5. Verificación de extremo a extremo

Igual que en el Capítulo 3, la verificación del flujo completo se apoya en dos comprobaciones independientes. La primera es el propio resultado de cada tarea del flujo de Airflow,

con sus recuentos de registros generados, aceptados y rechazados. La segunda es una lectura de los mismos datos desde un proceso Spark ajeno a la ejecución que los produjo, que debe coincidir exactamente con esos recuentos.

Una ejecución con los parámetros por defecto aterrizó 19 469 registros del subconjunto ORCID masivo en cuarenta fragmentos, junto con 531 registros rechazados, 1 000 documentos CVN sintéticos y 200 respuestas de la API de enriquecimiento, con todos los registros aterrizados portando los doce campos de su sobre de procedencia y sin ningún identificador de registro duplicado. Una lectura independiente con Spark de esos mismos objetos, ejecutada desde un pod distinto, contó exactamente esas mismas cifras. Esa misma lectura reveló un problema real de diseño: Spark lee todos los objetos bajo un prefijo, tenga o no fichero de manifiesto, así que una ejecución que había fallado el umbral de rechazo dejaba sus registros ya aterrizados visibles para cualquier lector que ignorase el manifiesto. La regla de que un fichero de manifiesto ausente señala una partición incompleta era, hasta entonces, solo una convención. La corrección aplicada borra los objetos válidos ya aterrizados de una ejecución que falla. La zona de rechazados de esa misma ejecución no se toca, y sus registros descartados quedan disponibles para diagnóstico igual que en cualquier ejecución que no falla. Una repetición de la misma prueba, tras el cambio, no dejó ningún objeto huérfano en el almacenamiento.

Esta forma de trabajar deja una limitación conocida y aceptada para este alcance. La semilla de generación sintética se mantiene fija entre ejecuciones para dar una base de comparación reproducible, así que cada ejecución diaria del flujo aterriza los mismos documentos sintéticos en una nueva partición por fecha, y corresponde a la transformación del Capítulo 5 deduplicar por identificador de registro o leer solo la partición más reciente.

5. Procesamiento distribuido: transformación, resolución de entidades e indicadores

El presente capítulo describe la parte computacionalmente más densa del sistema. Se recorren la validación y normalización de bronze a silver, la resolución de identidad entre un mismo investigador presente en ORCID y en un CVN sintético, y el cálculo de indicadores de investigación publicados en gold. Es el capítulo donde más se demuestra el procesamiento distribuido y donde el enlace por identificador ORCID sembrado en el Capítulo 4 rinde su fruto, al convertirse en la evidencia que permite fusionar los registros de una misma persona.

5.1. Arquitectura de la transformación

La transformación se implementa como tres trabajos de Spark, encadenados en un único flujo de Airflow llamado `transform_publish`. El primero, `bronze_to_silver`, valida y normaliza bronze a silver. El segundo, `silver_to_gold`, calcula los indicadores de investigación. El tercero, `publish_gold_to_postgres`, publica gold en una base de datos relacional. Cada uno corre en su propio pod, igual que la ejecución de Spark verificada en el Capítulo 3 y la ingesta del Capítulo 4. Una restricción nueva condiciona todo el código de este capítulo. La distribución de Spark empleada trae su propio intérprete de Python, en la versión 3.10, distinta de la versión 3.14 que exige el resto del proyecto. El código de este capítulo debe, por tanto, ser compatible con Python 3.10, comprobado mediante una prueba dedicada que analiza la sintaxis de cada módulo contra esa versión antes de ejecutarlo.

Las dos primeras etapas escriben directamente tablas Iceberg bajo los espacios de nombres `lakehouse.silver` y `lakehouse.gold`, sin aterrizaje intermedio de ficheros crudos, como quedó establecido en la Sección 3.4. La tercera etapa publica gold en una base de datos

PostgreSQL dedicada, para que el panel de visualización del Capítulo 6 nunca necesite consultar Iceberg ni Spark directamente. Las tres etapas comparten una única imagen de Spark, construida añadiendo a la imagen del Capítulo 3 las dependencias de validación del proyecto y el controlador JDBC de PostgreSQL. El número de ejecutores, la memoria de cada ejecutor y el número de particiones de mezcla de cada trabajo son parámetros explícitos del flujo. El Capítulo 6 reutiliza esos parámetros como variable de la prueba de rendimiento que allí se describe.

5.2. Validación y normalización de bronze a silver

El primer trabajo lee cada fuente de bronze por separado como líneas de texto, para preservar el documento original y evitar que la inferencia de esquema de Spark colapse el contenido heterogéneo de un CVN. Cada línea se deduplica por identificador de registro, conservando la copia con la fecha de aterrizaje más reciente, antes de analizar su contenido, así que ningún trabajador llega a analizar dos veces un documento repetido entre particiones de ingesta.

La validación de cada fuente se hace por capas, con severidad creciente. Un documento CVN pasa tres comprobaciones, cada una más estricta que la anterior:

- **Estructural.** Repite la validación del formato Open CVN ya usada en el aterrizaje del Capítulo 4, como defensa adicional frente a otro posible productor de bronze en el futuro.
- **Por entidad.** Valida cada entrada contra la definición estricta de su propio tipo de entidad, el mismo mecanismo que ya demostró ser necesario durante la generación sintética.
- **Reglas de negocio.** Comprueba reglas propias de esta etapa, como que todo identificador ORCID declarado tenga un dígito de control válido y que las fechas de cada periodo respeten su orden.

Un registro de ORCID pasa una validación paralela basada en reglas, con el dígito de control del identificador y la presencia de nombre y de al menos una afiliación u obra. Los registros de cualquiera de las dos fuentes que no superan sus reglas de validación se separan a una tabla de rechazados con la regla concreta que fallaron, siguiendo el mismo principio de conservar el rastro ya aplicado al aterrizaje en bronze, y el trabajo falla si la proporción de rechazados de una fuente supera un umbral del 5%.

Los registros válidos se normalizan a una forma común antes de la resolución de identidad, en dos frentes:

- **Nombres.** Se pliegan sin acentos y en minúsculas, y las partículas de apellido como «de» o «del» se descartan al construir la clave de bloqueo, para que «de la Torre» quede indexado por «torre» y no por la partícula.
- **Organizaciones.** Se reducen a su conjunto de palabras significativas, ordenado y sin las de relleno, de forma que el orden en que aparecen dentro del nombre deje de importar y pueda calcularse una similitud por solapamiento de conjuntos.

Sobre el subconjunto real verificado con 10 000 CVN sintéticos generados, esta etapa produjo 30 868 registros de persona válidos, 99 979 afiliaciones y 543 879 publicaciones, sin ningún registro rechazado.

5.3. Resolución de identidad entre fuentes

Fusionar el registro de un mismo investigador presente tanto en ORCID como en un documento CVN es el núcleo del problema de fusión planteado en el Capítulo 1. El estado del arte de la resolución de entidades ofrece, a grandes rasgos, tres familias de enfoque:

- **Coincidencia determinista por reglas explícitas.** Un identificador único compartido y, en su ausencia, nombre normalizado y afiliación compartida.
- **Vinculación probabilística de registros.** Calcula una probabilidad de coincidencia combinando varios campos con pesos calibrados, en lugar de una decisión binaria. La formalización clásica es de Fellegi y Sunter [Fellegi and Sunter, 1969]. Splink [UK Ministry of Justice, 2026] es una herramienta que la implementa.
- **Resolución basada en aprendizaje automático o representaciones vectoriales.** Aprende una función de similitud a partir de ejemplos etiquetados. La biblioteca `dedupe` [DataMade, 2026] es un ejemplo disponible.

La opción adoptada es la coincidencia determinista, en tres reglas aplicadas por orden de fuerza de evidencia:

- **Regla R1.** Fusiona todo registro que declare el mismo identificador ORCID válido, evidencia fuerte porque desambiguar autores de forma inequívoca es el objetivo fundacional de ORCID.
- **Regla R2.** La regla de repliegue para un CVN que no declara ese identificador. Exige que la clave de bloqueo del CVN, apellido normalizado más inicial del nombre, coincida con la de una entidad ya anclada por R1, que el nombre del CVN sea compatible

permitiendo relajar como mucho una de las dos partes, apellido o nombre, nunca las dos a la vez, que alguna organización de esa entidad coincida tras la normalización, y que exactamente una entidad cumpla las tres condiciones.

- **Regla R3.** Deja como entidad propia todo registro que ninguna de las dos anteriores fusiona.

La Figura 5.1 resume el orden de aplicación. El diseño favorece la precisión sobre la cobertura de forma deliberada. Una fusión equivocada corrompe todo indicador que dependa de esa fusión más adelante, mientras que una fusión no realizada solo divide una entidad en dos.

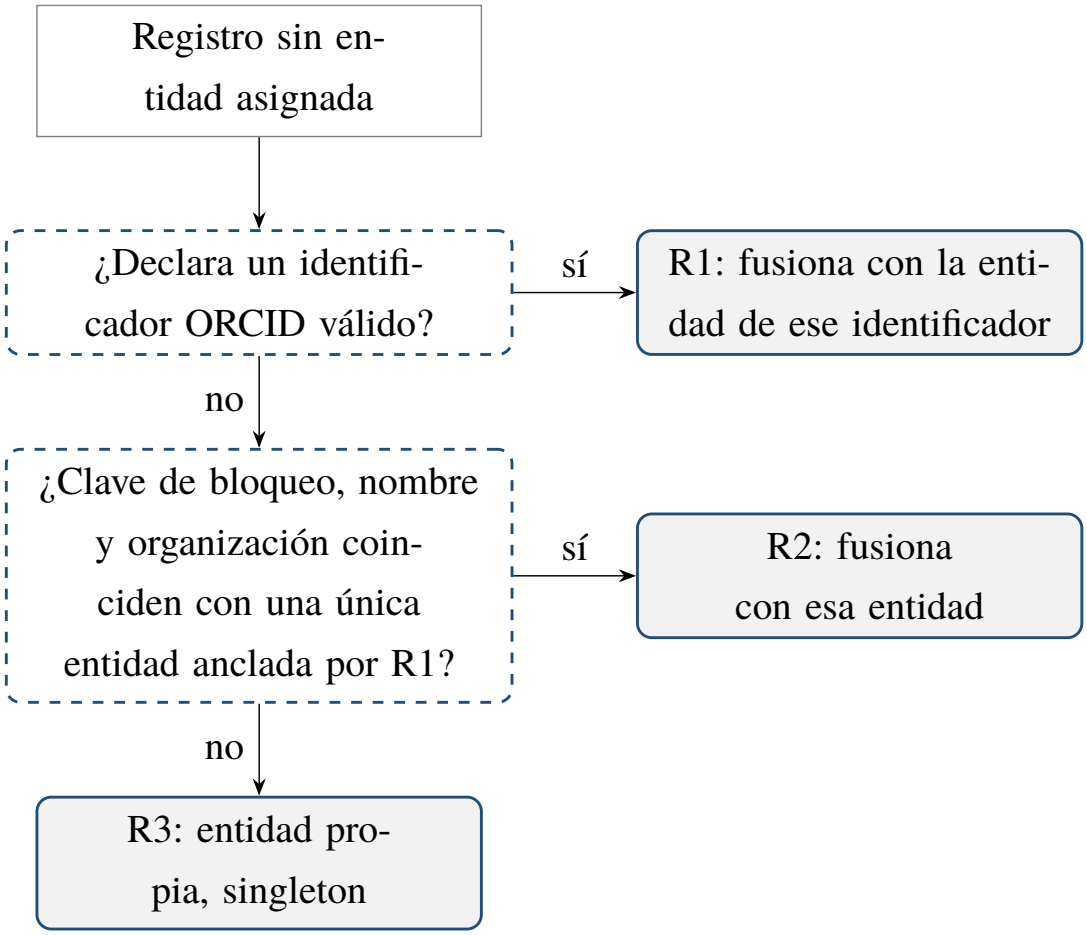


Figura 5.1: Orden de aplicación de las reglas de resolución de identidad.

Frente a la vinculación probabilística o el aprendizaje automático, un enfoque determinista con evidencia explícita por tipo de enlace produce resultados auditables campo a campo,

una propiedad valiosa en un trabajo académico donde cada decisión debe poder justificarse de forma directa. La alternativa probabilística o de aprendizaje automático queda documentada como trabajo futuro.

El resultado se midió contra un conjunto de verdad de referencia que el propio generador sintético produce, no contra una anotación externa, sobre la ejecución de 10 000 documentos CVN. La Tabla 5.1 resume el resultado de cada regla.

Regla	Evidencia	Evaluables	Correctas	Precisión	Cobertura
R1	Identificador ORCID compartido	7 098	7 098	100%	100%
R2	Nombre y afiliación normalizados	186	138	95,8%	74,2%

Tabla 5.1: Resolución de identidad sobre la ejecución de 10 000 documentos CVN.

Esa cobertura del 74,2% está en su techo real y no en un defecto de la regla. Una cuarta parte de los documentos evaluables no declara ninguna afiliación, y sin afiliación la regla no tiene evidencia posible por diseño. Un umbral de precisión del 99% se había fijado como objetivo antes de medir, y no se alcanza. Exigir más de una organización compartida para aceptar un nombre acortado sube la precisión al 98,5% pero baja la cobertura al 70,4%, y ningún ajuste del umbral de similitud entre organizaciones cambia el resultado. Los seis falsos positivos de la tabla comparten nombre y una única organización con la persona correcta, indistinguibles con esta evidencia de una coincidencia real. La cifra que se retiene, por tanto, es la de la tabla, aceptada como el límite honesto de este diseño en lugar de perseguirse artificialmente con más reglas.

5.4. Cálculo de indicadores y publicación en PostgreSQL

El segundo trabajo calcula tres indicadores de investigación sobre silver. El primero cuenta publicaciones por investigador y año. El segundo construye la línea temporal de afiliaciones de cada entidad, con su rango de años de carrera. El tercero detecta pares de colaboración entre entidades que comparten el identificador de una misma obra. Antes de calcular el primero, es necesario deduplicar las publicaciones por entidad, porque una misma obra puede aparecer

en varios registros fusionados en una entidad, un CVN y su registro ORCID enlazado, por ejemplo. Sobre los datos reales verificados, deduplicar redujo 543 879 filas de publicación a 516 396 obras distintas. Sin esa deduplicación, el indicador habría contado un 5,1% de publicaciones de más.

El indicador de colaboración exige una comprobación previa, porque el mismo problema de cobertura de la Sección 5.3 puede producir pares de colaboración falsos. Una persona cuyo CVN no llegó a fusionarse con su propio registro ORCID reportaría las mismas obras desde dos entidades distintas, y aparecería colaborando consigo misma. Sobre los 9 540 pares de entidades que comparten alguna obra en los datos reales, solo 25, un 0,26%, resultaron ser ese artefacto, medido contra el mismo conjunto de verdad de referencia de la sección anterior. El indicador se construye igualmente, con una marca por par que indica si alguna de las dos entidades procede de un CVN, para poder restringir el análisis a colaboraciones solo entre datos reales de ORCID cuando se desee esa lectura más estricta.

Gold se publica en PostgreSQL a través de tablas de preproducción y un intercambio atómico. El trabajo crea cada tabla final vacía con su definición explícita, carga sus filas mediante una escritura JDBC ordinaria, y solo entonces sustituye la tabla publicada por la nueva dentro de una única transacción, de forma que el panel de visualización nunca ve una tabla a medio cargar. Antes de esa sustitución se reconcilia el número de filas entre Iceberg y PostgreSQL, y cualquier discrepancia aborta la publicación sin tocar lo ya publicado. Este mecanismo se verificó deliberadamente matando el proceso de publicación a mitad de carga. Las tablas ya publicadas permanecieron intactas, y una repetición posterior del mismo trabajo limpió los restos de la carga interrumpida y completó la publicación con normalidad.

5.5. Verificación de extremo a extremo

Siguiendo el mismo patrón de doble comprobación de los capítulos anteriores, el resultado de cada trabajo se contrastó con un cálculo independiente. Para gold, ese cálculo fue un script en Python normal, escrito deliberadamente con estructuras de datos distintas a las de Spark, que recalculó cada tabla a partir de una exportación de silver. Comparado fila a fila contra el resultado real del flujo, no encontró ninguna discrepancia sobre 306 599 filas publicadas en PostgreSQL. Una segunda ejecución completa del flujo, sobre el mismo bronze, produjo un contenido idéntico fila a fila en PostgreSQL, lo que confirma que la reconstrucción completa en cada ejecución es determinista.

La última comprobación midió cuánto dependen los indicadores de los errores conocidos de la resolución de identidad. Deshacer las 75 fusiones de la regla R2 que se apoyan en

una única organización compartida, el subconjunto de evidencia más débil, desplaza cada indicador como mucho un 0,58%. Los indicadores de gold son, por tanto, insensibles en la práctica a la precisión del 95,8% medida para la regla R2, aunque esa precisión no alcance el objetivo inicial del 99%.

6. Visualización, evaluación de rendimiento y endurecimiento

El presente capítulo cierra la demostración del sistema con tres piezas. Se recorren el panel de indicadores como resultado visible de toda la plataforma, el banco de pruebas de rendimiento como evaluación cuantitativa formal del procesamiento distribuido, y el endurecimiento como evidencia de que el sistema se sometió a uso real y no solo a una demostración controlada. El capítulo termina con la reconstrucción completa de la plataforma desde cero, que cierra el objetivo de reproducibilidad planteado en el Capítulo 1.

6.1. Panel de indicadores

El panel se construye con Apache Superset [Apache Software Foundation, 2026g], conectado a la base de datos PostgreSQL de la capa *gold* descrita en el Capítulo 5, nunca a Iceberg ni a Spark directamente. La conexión usa un rol de solo lectura dedicado, distinto del rol que publica los datos, con permiso de lectura sobre las tablas existentes y sobre cualquier tabla futura del mismo esquema. Esa segunda condición es necesaria porque la publicación de gold sustituye sus tablas en cada ejecución del flujo, y un permiso otorgado solo sobre las tablas actuales se perdería en la primera sustitución.

El panel muestra siempre agregados, nunca el nombre de una persona concreta, aunque la tabla de dimensión de investigadores sí contiene nombres reales tomados de ORCID. Ningún gráfico del panel consulta esa tabla. El panel se define como código versionado, exportado a un paquete reproducible en lugar de dejarse como configuración manual solo accesible desde la interfaz. Esa exportación se verificó de forma directa. Tras borrar por completo el panel, sus gráficos, sus conjuntos de datos y su conexión, reimportar el paquete exportado

reprodujo exactamente el mismo panel, con el mismo identificador interno y con cada gráfico devolviendo las mismas filas que antes del borrado.

El panel, mostrado en la Figura 6.1, reúne cuatro elementos:

- **Publicaciones por año.** El primer indicador de investigación, agregado por año de publicación.
- **Línea temporal de afiliaciones.** El tercer indicador, agregado por año de inicio y tipo de periodo.
- **Pares de colaboración.** El segundo indicador, restringido a los pares que involucran solo datos reales de ORCID, sin ningún miembro procedente de un CVN sintético.
- **Procedencia.** Una tabla con el identificador de la última ejecución del flujo de transformación y sus recuentos, para que cada cifra del panel pueda atribuirse a una ejecución concreta.



Figura 6.1: Panel de indicadores de investigación en Apache Superset, con los tres indicadores y la tabla de procedencia de la ejecución.

La conexión del panel deshabilita además la caché de resultados, para que una nueva ejecución del flujo de transformación se refleje de inmediato sin necesidad de vaciarla a mano. Esto se verificó ejecutando el flujo de transformación completo del Capítulo 5 mientras un proceso externo consultaba el panel de forma repetida. A lo largo de 1 040 segundos y 238 consultas no se encontró ningún error ni ninguna respuesta vacía. El identificador de ejecución

mostrado por el panel pasó de la ejecución anterior a la nueva en un único paso, sin ningún estado intermedio visible, mientras el resto de cifras permanecía igual, como corresponde a una transformación determinista sobre los mismos datos de origen.

6.2. Metodología y resultados del banco de pruebas de rendimiento

El banco de pruebas mide el efecto del número de ejecutores de Spark sobre el tiempo de los tres trabajos de transformación del Capítulo 5, a tres escalas de volumen de datos y con tres cantidades de ejecutores en cada una. La campaña se aísla por completo de los datos de producción, con su propio cubo de MinIO, sus propios espacios de nombres de Iceberg y su propio esquema de PostgreSQL por escala, de forma que ninguna medición pueda alterar el panel del Capítulo 5 ni sus propios datos.

Cada combinación de escala y número de ejecutores se ejecuta varias veces. La primera ejecución de cada escala se descarta como calentamiento, porque leer un volumen de datos por primera vez resultó sistemáticamente más lento que las ejecuciones siguientes. Las ejecuciones que sí cuentan se repiten tres veces, intercaladas entre configuraciones en lugar de agrupadas, para que una deriva del propio equipo durante la campaña no se confunda con un efecto del número de ejecutores. La corrección de cada resultado se comprueba además con una huella de contenido de las tablas producidas, calculada de forma independiente de su orden. Dos configuraciones distintas que terminan con éxito sobre la misma escala deben producir exactamente la misma huella, y así ocurrió en todos los casos comparables de la campaña.

La captura de estas métricas no necesitó desplegar una pila de observabilidad dedicada como Prometheus [Prometheus Authors, 2026] junto con Grafana [Grafana Labs, 2026]. Cada ejecución de Spark ya escribe su propio registro de eventos, con la duración de cada etapa y tarea y el uso de cada ejecutor, y ese registro basta para una campaña acotada como esta. Instalar y operar una pila de observabilidad continua solo para esta campaña puntual no estaba justificado, y queda documentada como trabajo futuro para un despliegue en producción.

El hallazgo principal es que el número de ejecutores no es solo una palanca de velocidad. A partir de cierto volumen, se convierte en una palanca de fiabilidad. Con el dimensionamiento de memoria fijo por ejecutor empleado en esta campaña, un único ejecutor deja de poder completar el trabajo de validación y normalización de forma determinista a partir del doble del volumen base, agotando su memoria en cada intento. A cuádruple volumen, ni siquiera dos ejecutores bastan para ese mismo trabajo, y solo se completa con cuatro. La Figura 6.2

muestra este comportamiento sobre el volumen base, donde el trabajo sí completa a cualquier número de ejecutores.

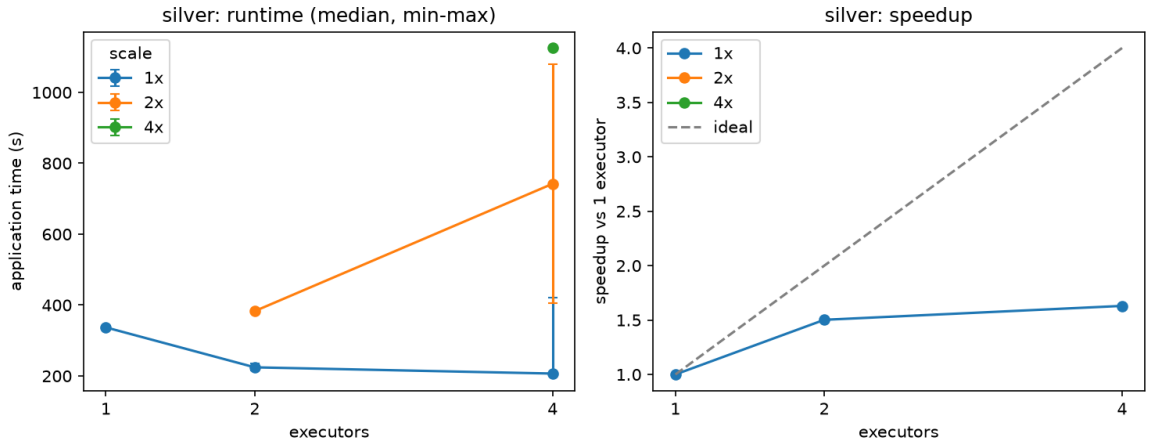


Figura 6.2: Tiempo de ejecución y aceleración del trabajo de validación y normalización, sobre el volumen de datos base.

Incluso cuando el trabajo sí completa, la aceleración queda lejos de la ideal. Sobre el volumen base, pasar de uno a dos ejecutores da una aceleración de 1,50, y pasar a cuatro solo sube esa cifra a 1,63, con una eficiencia que cae de 0,75 a 0,41. Los otros dos trabajos de la transformación se comportan de forma todavía más marcada. El cálculo de indicadores ve su eficiencia caer de 1,00 a 0,53 y a 0,23 al pasar de uno a dos y a cuatro ejecutores, en cualquiera de las tres escalas, porque su volumen de datos, ya agregado, es demasiado pequeño para aprovechar más paralelismo. La publicación en PostgreSQL apenas crece con el volumen de datos en absoluto, con un tiempo de entre 61 y 142 segundos en cualquier combinación de escala y número de ejecutores, frente a un crecimiento prácticamente lineal del trabajo de validación con el volumen. La Tabla 6.1 resume esta diferencia mediante el exponente de la relación entre tiempo y volumen de datos, donde un valor de 1 indica crecimiento lineal y un valor negativo indica que el tiempo no depende del volumen.

Trabajo	1 ejecutor	2 ejecutores	4 ejecutores
Validación y normalización	–	0,77	1,22
Cálculo de indicadores	0,97	0,96	0,79
Publicación en PostgreSQL	-0,09	-0,15	-0,16

Tabla 6.1: Exponente de crecimiento del tiempo de ejecución frente al volumen de datos, por trabajo y número de ejecutores.

Ninguna de las tres etapas satura por falta de procesador. El uso de sistema nunca superó el 32,2% en ninguna ejecución, y ese máximo se dio precisamente con cuatro ejecutores, el mismo punto donde el uso del procesador del propio servicio de almacenamiento de objetos también alcanzó su máximo. Este patrón apunta al almacenamiento de objetos compartido, y no al procesador, como el límite más probable de la plataforma en un único nodo. Esa lectura es consistente con los datos disponibles, pero no queda demostrada de forma concluyente, ya que el mecanismo exacto dentro del servicio de almacenamiento no se aisló más allá de esta medición.

6.3. Endurecimiento: fallos operativos reales encontrados y corregidos

A diferencia de las limitaciones de diseño aceptadas de antemano en capítulos anteriores, esta sección recoge fallos reales que aparecieron usando el sistema completo bajo condiciones reales, no en una demostración controlada. Encontrar y corregir un fallo de un componente externo mediante diagnóstico y una recuperación operativa reproducible es, en sí mismo, un resultado de este trabajo.

- **Bloqueo permanente del orquestador de flujos.** Tras un periodo de funcionamiento sostenido, el propio servidor de tareas de Airflow quedó incapaz de arrancar, fallando de la misma forma en cada reinicio. La causa es un error conocido, sin corregir todavía en la versión de Airflow empleada, que se dispara al reconciliar una tarea huérfana cuyo pod ya no existe. El registro de esa tarea concreta quedaba almacenado sin marcarse como terminado, así que cada arranque del servidor volvía a intentar reconciliarla y volvía a fallar de la misma manera. Al tratarse de un error sin corrección disponible en la propia herramienta, la solución fue operativa en lugar de un cambio de código.

Marcar manualmente esa tarea como fallida y reiniciar el servidor de tareas resolvió el bloqueo de inmediato. Esta misma incidencia volvió a producirse, de forma independiente, semanas después, y la misma receta operativa la resolvió de nuevo sin ningún cambio adicional.

- **Cierre del servidor de la interfaz de Airflow bajo carga.** Ese mismo servidor carecía de una reserva explícita de procesador y memoria, lo que le asignaba la prioridad de planificación más baja de todo el clúster. Bajo carga del propio nodo, quedaba desatendido el tiempo suficiente para que su propia comprobación de salud lo diera por bloqueado y lo reiniciase, interrumpiendo cualquier flujo que dependiera de él en ese momento. Se corrigió fijando una reserva explícita de recursos para ese servidor y relajando los tiempos de su comprobación de salud. La corrección se verificó en el propio clúster momentos después de que el fallo original volviera a producirse de forma independiente, confirmando que el nuevo ajuste resolvía exactamente ese mismo patrón.

6.4. Verificación de extremo a extremo: reconstrucción desde cero

La plataforma se diseñó para poder reconstruirse desde cero siguiendo únicamente una guía escrita, sin pasos manuales no documentados. Esa guía se validó en la práctica y no solo se redactó. La reconstrucción completa se hizo sobre un clúster Kubernetes aislado y desechable, levantado con k3d [k3d Authors, 2026], en lugar de destruir el clúster real que ya contenía los datos de producción de los capítulos anteriores. El clúster aislado se configuró con la misma versión exacta de Kubernetes que el clúster real, y con el propio repositorio de código montado en la misma ruta, de forma que la guía se siguiera sin ningún paso distinto al que seguiría un lector cualquiera.

Esta reconstrucción encontró un fallo real no observado hasta entonces. Al arrancar todos los servicios base a la vez sobre un clúster completamente nuevo, el primer arranque de PostgreSQL quedó interrumpido por la contención de arrancar varios servicios a la vez, antes de llegar a crear el usuario que la capa *gold* necesita. La imagen empleada no reintenta esa inicialización interrumpida en arranques posteriores, así que el servicio quedaba disponible pero permanentemente incapaz de autenticar ese usuario. El fallo se diagnosticó revisando el registro del propio contenedor y se corrigió forzando una reinicialización completa de ese servicio. Con esa corrección aplicada, la plataforma completa, incluidos ambos flujos de Airflow y el panel de Superset con su propio contenido importado, funcionó de extremo a extremo

sobre el clúster recién creado. El clúster aislado se eliminó al terminar, sin que el clúster real ni sus datos de producción se vieran afectados en ningún momento del proceso.

7. Conclusiones, competencias y trabajo futuro

El Capítulo 1 planteó el problema de partida y un objetivo general acompañado de nueve objetivos específicos. Los capítulos intermedios han desplegado, capa a capa, la plataforma que responde a ese planteamiento, y el Capítulo 6 ha aportado evidencia reproducible de su comportamiento. Este capítulo cierra el documento retomando esos objetivos, relacionando el trabajo con los resultados de aprendizaje del máster, y señalando las limitaciones y el trabajo futuro que se derivan de todo lo anterior.

7.1. Resumen del trabajo

Problema y motivación. La información curricular y de trayectoria investigadora está dispersa entre documentos individuales, como CVN, y registros públicos a escala masiva, como ORCID, sin una plataforma abierta y autoalojada que combine ambas naturalezas de dato, tal y como se planteó en el Capítulo 1.

Arquitectura desplegada. Este trabajo construye una plataforma *lakehouse*, justificada en el Capítulo 2 y desplegada en el Capítulo 3: Kubernetes, MinIO con Iceberg, Spark orquestado por Airflow y PostgreSQL. El Capítulo 4 ingiere y fusiona ORCID real y CVN sintético, y el Capítulo 5 resuelve su identidad y calcula los indicadores.

Resultados. El Capítulo 6 publica esos indicadores en un panel verificado en directo, mide el efecto del paralelismo sobre el rendimiento, y documenta fallos operativos reales corregidos, incluida una reconstrucción completa desde cero.

7.2. Cumplimiento de los objetivos

El objetivo general planteado en el Capítulo 1 era diseñar, desplegar y evaluar una plataforma que integrase CVN sintético y ORCID real, los procesase con resolución de identidad entre fuentes, y publicase indicadores en un panel evaluado mediante un banco de pruebas de rendimiento. Se considera cumplido. La Tabla 7.1 detalla cada objetivo específico, en el mismo orden del Capítulo 1.

Uno de los objetivos se cumple con garantías parciales, de forma medida y no oculta: la integración de ORCID con CVN alcanza una precisión del 95,8% y una cobertura del 74,2% en su regla de repliegue por nombre y afiliación, por debajo del objetivo de precisión fijado antes de medir. El Capítulo 5 explicó por qué perseguirla artificialmente con más reglas empeoraría, no mejoraría, la fiabilidad del sistema.

Obj.	**Descripción resumida**	**Cumplimiento**
OE1	Despliegue de la infraestructura base de la plataforma	Cumplido
OE2	Validación del mecanismo central de almacenamiento y procesamiento	Cumplido
OE3	Incorporación de datos ORCID reales	Cumplido
OE4	Incorporación de currículos CVN validados frente a Open CVN	Cumplido
OE5	Integración de ORCID con CVN por medio de Open CVN	Cumplido con garantías parciales
OE6	Cálculo y publicación consistente de indicadores	Cumplido
OE7	Visualización de los indicadores en un panel accesible	Cumplido
OE8	Banco de pruebas de rendimiento por paralelismo y escala	Cumplido
OE9	Robustez operativa mediante uso real y guía de reproducción	Cumplido

Tabla 7.1: Grado de cumplimiento de los objetivos específicos.

7.3. Contribuciones principales

Las contribuciones de este trabajo quedan demostradas, con evidencia concreta, a lo largo del documento:

- **Plataforma *lakehouse* verificada de extremo a extremo**: el Capítulo 3 demuestra con dos comprobaciones independientes que el mecanismo central de almacenamiento y procesamiento funciona antes de construir ningún flujo de negocio sobre él.
- **Mecanismo de fusión entre ORCID y CVN diseñado desde el origen del dato**: el Capítulo 4 siembra los currículos sintéticos con el identificador ORCID de su semilla real desde su propia generación, sin enlazar dos fuentes independientes a posteriori.
- **Resolución de identidad determinista con evidencia auditable**: el Capítulo 5 fusiona registros de fuentes distintas mediante reglas explícitas, cada una con su propia evidencia, y mide su precisión y su cobertura.
- **Hallazgo del banco de pruebas de rendimiento**: el Capítulo 6 demuestra que el número de ejecutores de Spark es una palanca de fiabilidad, no solo de velocidad.
- **Guía de reproducibilidad validada por una reconstrucción real**: la plataforma se reconstruyó desde cero sobre un clúster aislado siguiendo únicamente esa guía, en el Capítulo 6.

7.4. Resultados de aprendizaje desarrollados

El Capítulo 1 identificó seis resultados de aprendizaje del máster trabajados en este proyecto. La Tabla 7.2 recoge su evidencia concreta.

Resultado	Evidencia concreta
CN02	Arquitectura *lakehouse* justificada en el Capítulo 2, desplegada en el Capítulo 3.
HA01	Adquisición y generación de fuentes enlazadas por diseño en el Capítulo 4, fusionadas en el Capítulo 5.
HA02	Procesamiento distribuido del Capítulo 5, evaluado cuantitativamente en el Capítulo 6.
HA03	Orquestación de flujos ETL de los Capítulos 3 y 5, con su almacenamiento escalable.
CP01	Planificación y despliegue de una solución Big Data, de los Capítulos 2, 3 y 6.
CP04	Proyecto profesional original de la plataforma completa, evaluada en el Capítulo 6 y cerrada aquí.

Tabla 7.2: Resultados de aprendizaje del máster y evidencia concreta de cada uno.

7.5. Limitaciones y trabajo futuro

Los capítulos anteriores han documentado limitaciones concretas sin ocultarlas, distinguiendo las que proceden de un componente de terceros de las que son una decisión deliberada de alcance. Esta sección las consolida junto con el trabajo futuro que se deriva de cada una.

La única de origen externo es el **fallo conocido del orquestador de flujos**, sin corrección oficial disponible, documentado con su receta operativa de recuperación en el Capítulo 6. El trabajo futuro solo puede consistir en incorporar la corrección oficial cuando exista.

El resto son decisiones deliberadas de alcance, cada una ya justificada en su capítulo:

- sin capa de consulta interactiva independiente como Trino [Trino Software Foundation, 2026]
- sin observabilidad centralizada como Prometheus [Prometheus Authors, 2026] y Grafana [Grafana Labs, 2026]
- un único nodo local, sin infraestructura como código mediante Terraform [HashiCorp, Inc., 2026], en vez de un clúster real en la nube
- reconstrucción completa de silver y gold en cada ejecución, en lugar de procesamiento incremental
- resolución de identidad determinista, frente a las alternativas probabilísticas ya citadas en el Capítulo 5

La Tabla 7.3 consolida cada una con su trabajo futuro asociado.

Limitación	Trabajo futuro asociado
Fallo del orquestador de flujos, sin corrección oficial	Incorporar la corrección oficial cuando esté disponible
Sin capa de consulta interactiva independiente	Incorporar un motor de consulta federado si surge esa necesidad
Sin observabilidad centralizada	Desplegar una pila de observabilidad continua para producción
Único nodo local, sin infraestructura como código para la nube	Automatizar el despliegue en la nube sobre la misma definición
Reconstrucción completa en cada ejecución	Procesar de forma incremental sobre los *snapshots* de Iceberg
Resolución de identidad determinista, sin aprendizaje automático	Explorar vinculación probabilística o basada en aprendizaje automático

Tabla 7.3: Limitaciones documentadas y trabajo futuro asociado.

7.6. Conclusiones

El resumen, el cumplimiento de objetivos y las contribuciones ya presentados permiten cerrar el documento con una valoración conjunta. El objetivo general se considera cumplido y, con él, los nueve objetivos específicos, con la única salvedad medida de la cobertura de la resolución de identidad, una limitación de evidencia y no una afirmación sin respaldo. Las limitaciones recogidas no comprometen esta conclusión. Delimitan, con precisión y sin ocultamiento, hasta dónde llega la propuesta actual. Algunas son ajenas al sistema desarrollado y otras son decisiones deliberadas de alcance, propias de un Trabajo Fin de Máster y no de un producto cerrado, y el sistema las documenta en lugar de disimularlas.

En conjunto, este trabajo demuestra que es posible construir, a partir de CVN y ORCID, una plataforma *lakehouse* autoalojada que fusiona información curricular documental con registros públicos a escala masiva, conserva procedencia verificable en cada capa, y ofrece garantías reproducibles allí donde el proceso es determinista. Esa plataforma completa, más que el panel de indicadores o cualquier componente aislado que la demuestra, es la aportación principal de este Trabajo Fin de Máster.

Referencia bibliográfica

[Anyscale, Inc., 2026] Anyscale, Inc. (2026). Ray. https://www.ray.io/. Accedido: 2026-09-29.

[Apache Software Foundation, 2026a] Apache Software Foundation (2026a). Apache airflow providers – CNCF Kubernetes (KubernetesExecutor). https://airflow.apache.org/docs/apache-airflow-providers-cncf-kubernetes/stable/index.html. Accedido: 2026-09-28.

[Apache Software Foundation, 2026b] Apache Software Foundation (2026b). Apache flink: Stateful computations over data streams. https://flink.apache.org/. Accedido: 2026-09-29.

[Apache Software Foundation, 2026c] Apache Software Foundation (2026c). Apache hadoop. https://hadoop.apache.org/. Accedido: 2026-09-29.

[Apache Software Foundation, 2026d] Apache Software Foundation (2026d). Apache hive: Design – metastore. https://cwiki.apache.org/confluence/display/hive/design. Accedido: 2026-09-29.

[Apache Software Foundation, 2026e] Apache Software Foundation (2026e). Apache hudi. https://hudi.apache.org/. Accedido: 2026-09-28.

[Apache Software Foundation, 2026f] Apache Software Foundation (2026f). Apache iceberg documentation. https://iceberg.apache.org/. Accedido: 2026-09-28.

[Apache Software Foundation, 2026g] Apache Software Foundation (2026g). Apache superset. https://superset.apache.org/. Accedido: 2026-09-28.

[Apache Software Foundation, 2026h] Apache Software Foundation (2026h). Running spark on Kubernetes. https://spark.apache.org/docs/latest/running-on-kubernetes.html. Accedido: 2026-09-28.

[Armbrust et al., 2021] Armbrust, M., Ghodsi, A., Xin, R., and Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *11th Annual Conference on Innovative Data Systems Research (CIDR '21)*, Online. 11–15 de enero de 2021.

[Broadcom Inc. / Bitnami Charts Contributors, 2025] Broadcom Inc. / Bitnami Charts Contributors (2025). Regarding the bitnami deprecation and migration plan (issue #35164). https://github.com/bitnami/charts/issues/35164. Accedido: 2026-09-29.

[Cloud Native Computing Foundation, 2026] Cloud Native Computing Foundation (2026). Kubernetes. https://www.cncf.io/projects/kubernetes/. Accedido: 2026-09-28. Proyecto graduado de la CNCF desde el 6 de marzo de 2018.

[Dagster Labs, 2026] Dagster Labs (2026). Dagster. https://dagster.io/. Accedido: 2026-09-29.

[Dask Development Team, 2026] Dask Development Team (2026). Dask: Scale the Python tools you love. https://www.dask.org/. Accedido: 2026-09-29.

[DataMade, 2026] DataMade (2026). dedupe: A python library for accurate and scalable fuzzy matching, record deduplication and entity resolution. https://docs.dedupe.io/en/latest/. Accedido: 2026-09-28.

[Delta Lake Project, Linux Foundation, 2026] Delta Lake Project, Linux Foundation (2026). Delta lake. https://delta.io/. Accedido: 2026-09-28.

[Docker, Inc., 2026] Docker, Inc. (2026). Swarm mode overview. https://docs.docker.com/engine/swarm/. Accedido: 2026-09-29.

[euroCRIS, 2026] euroCRIS (2026). Main features of CERIF. https://eurocris.org/services/main-features-cerif/. Accedido: 2026-09-28.

[Fellegi and Sunter, 1969] Fellegi, I. P. and Sunter, A. B. (1969). A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210.

[Figshare, 2026] Figshare (2026). Figshare: Online open access research data repository. https://figshare.com/. Accedido: 2026-09-29.

[Grafana Labs, 2026] Grafana Labs (2026). Grafana OSS. `https://grafana.com/oss/grafana/`. Accedido: 2026-09-29.

[HashiCorp, Inc., 2026] HashiCorp, Inc. (2026). Terraform. `https://www.terraform.io/`. Accedido: 2026-09-30.

[Jaén, 2026] Jaén, C. M. (2026). Especificación de formato y desarrollo de herramientas para el procesamiento de currículum vitae en el ámbito académico universitario. Trabajo fin de grado, Universidad de Castilla-La Mancha, Escuela Superior de Ingeniería Informática de Albacete.

[JSON Schema Organization, 2026] JSON Schema Organization (2026). JSON schema: A media type for describing JSON documents. `https://json-schema.org/`. Accedido: 2026-09-29.

[k3d Authors, 2026] k3d Authors (2026). k3d: k3s in Docker. `https://k3d.io/`. Accedido: 2026-09-30.

[K3s Project Authors, 2026] K3s Project Authors (2026). k3s: Lightweight Kubernetes. `https://docs.k3s.io/`. Accedido: 2026-09-28. Proyecto originado en Rancher Labs (SUSE).

[Kubeflow, 2026] Kubeflow (2026). Kubernetes operator for apache spark. `https://github.com/kubeflow/spark-operator`. Accedido: 2026-09-29.

[Metabase, Inc., 2026] Metabase, Inc. (2026). Metabase. `https://www.metabase.com/`. Accedido: 2026-09-29.

[Microsoft, 2026] Microsoft (2026). Power BI. `https://www.microsoft.com/en-us/power-platform/products/power-bi`. Accedido: 2026-09-29.

[MinIO, Inc., 2026] MinIO, Inc. (2026). MinIO: High-performance, S3 compatible object store. `https://github.com/minio/minio`. Accedido: 2026-09-28.

[ORCID, 2025] ORCID (2025). ORCID public data file 2025. `https://doi.org/10.6084/m9.figshare.30375589`. Accedido: 2026-09-28. Volumen verificado empíricamente en este proyecto: 26 078 951 registros escaneados (‘docs/roadmap/tfm/issues/issue-95-orcid-bulk-data-file-pipeline.md‘).

[ORCID, 2026a] ORCID (2026a). ORCID public API. `https://info.orcid.org/documentation/features/public-api/`. Accedido: 2026-09-29.

[ORCID, 2026b] ORCID (2026b). What is ORCID? https://info.orcid.org/what-is-orcid/. Accedido: 2026-09-28.

[Prefect Technologies, Inc., 2026] Prefect Technologies, Inc. (2026). Prefect: Workflow orchestration for data, ML and agents. https://www.prefect.io/. Accedido: 2026-09-29.

[Project Nessie, 2026] Project Nessie (2026). Project nessie: Transactional catalog for data lakes with Git-like semantics. https://projectnessie.org/. Accedido: 2026-09-28.

[Prometheus Authors, 2026] Prometheus Authors (2026). Prometheus: Monitoring system & time series database. https://prometheus.io/. Accedido: 2026-09-30.

[Salesforce, Inc., 2026] Salesforce, Inc. (2026). Tableau. https://www.tableau.com/. Accedido: 2026-09-29.

[Spotify, 2026] Spotify (2026). Luigi. https://github.com/spotify/luigi. Accedido: 2026-09-29.

[The Helm Authors, 2026] The Helm Authors (2026). Helm: The package manager for Kubernetes. https://helm.sh/. Accedido: 2026-09-29.

[The PostgreSQL Global Development Group, 2026] The PostgreSQL Global Development Group (2026). PostgreSQL. https://www.postgresql.org/. Accedido: 2026-09-29.

[Trino Software Foundation, 2026] Trino Software Foundation (2026). Trino: Distributed SQL query engine for big data. https://trino.io/. Accedido: 2026-09-29.

[UK Ministry of Justice, 2026] UK Ministry of Justice (2026). Splink: Probabilistic record linkage at scale. https://moj-analytical-services.github.io/splink/. Accedido: 2026-09-28.

[VIVO Project, 2026] VIVO Project (2026). VIVO: Connect, share, discover. https://www.vivoweb.org/. Accedido: 2026-09-28.

[Zaharia et al., 2016] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., and Stoica, I. (2016). Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.

Anexos

Los siguientes anexos recogen información de referencia complementaria al cuerpo principal del documento, demasiado extensa para incluirse en los capítulos anteriores. Detallan los pasos para reproducir la plataforma, los resultados completos del banco de pruebas, el modelo de datos de cada capa, las reglas de resolución de identidad con su análisis de errores, el registro de limitaciones y la organización del repositorio.

Anexo A: Guía de reproducibilidad de la plataforma

Este anexo recoge los pasos con los que se reconstruyó la plataforma desde cero en el Capítulo 6. No introduce ninguna decisión de diseño nueva. Es una referencia operativa que complementa con la sintaxis exacta de los comandos el despliegue descrito en el Capítulo 3, y su versión completa y mantenida vive en `docs/development/tfm_lakehouse_workflow.md`.

A.1. Prerrequisitos y versiones fijadas

La plataforma necesita WSL2 con systemd activado, Docker, `kubectl`, `helm` y `uv` para el entorno Python del propio repositorio. Todas las versiones de los servicios se fijan de forma explícita, de modo que dos despliegues distintos produzcan el mismo sistema. La Tabla 7.4 las reúne.

Componente	Versión fijada	Origen
Kubernetes con k3s	v1.36.4+k3s1	Instalador oficial de k3s
MinIO	*chart* 17.0.21	Referencia OCI de Bitnami
PostgreSQL	*chart* 18.11.3	Referencia OCI de Bitnami
Airflow	*chart* 1.22.0	Repositorio de Apache Airflow
Superset	*chart* 0.22.8, aplicación 6.1.0	Repositorio de Apache Superset
Spark	3.5.9	Distribución oficial con Hadoop 3
Iceberg	*runtime* 1.11.0 para Spark 3.5	Maven Central
Acceso a S3	`hadoop-aws` 3.3.4 y SDK 1.12.262	Maven Central

Tabla 7.4: Versiones fijadas de los componentes desplegados.

Las rutas de código y de datos se montan en los pods desde el directorio de trabajo del nodo. Por eso los dos ficheros de flujo contienen la ruta absoluta del repositorio en la máquina de desarrollo, y esa ruta debe editarse antes de entregarlos desde cualquier otra máquina. Es una carencia de portabilidad conocida, recogida en el Anexo D.8.

A.2. Clúster y servicios base

El Código A.1 instala k3s con el modo de red `host-gw`, necesario bajo WSL2, y crea el espacio de nombres dedicado. Los servicios se instalan después con los valores versionados bajo `infra/helm-values/`, tras crear sus credenciales como secretos de Kubernetes que nunca se guardan en el repositorio.

```
curl -sfL https://get.k3s.io | \
  INSTALL_K3S_EXEC="--flannel-backend=host-gw␣--write-kubeconfig-mode=644
     " sh -
kubectl create namespace tfm-lakehouse

helm install minio oci://registry-1.docker.io/bitnamicharts/minio \
  --version 17.0.21 -f infra/helm-values/minio-values.yaml -n tfm-
     lakehouse
helm install postgresql oci://registry-1.docker.io/bitnamicharts/
    postgresql \
  --version 18.11.3 -f infra/helm-values/postgresql-values.yaml -n tfm-
     lakehouse
helm install airflow apache-airflow/airflow \
  --version 1.22.0 -f infra/helm-values/airflow-values.yaml -n tfm-
     lakehouse
```

Código A.1: Instalación del clúster y de los tres servicios base.

A.3. Imágenes de Spark y de ingesta

El flujo de transformación ejecuta sus tres trabajos sobre una única imagen, construida por capas a partir de la imagen oficial de Spark. El Código A.2 muestra la construcción y la importación de esa imagen y de la imagen de ingesta. La importación en k3s es el único paso que exige privilegios de administrador.

```
docker build -f infra/spark-conf/Dockerfile.gold \
  -t tfm-lakehouse/spark-py:3.5.9-iceberg1.11.0-gold .
docker save tfm-lakehouse/spark-py:3.5.9-iceberg1.11.0-gold | \
  sudo k3s ctr images import -

docker build -f infra/ingest/Dockerfile -t tfm-lakehouse/ingest:py3.14 .
docker save tfm-lakehouse/ingest:py3.14 | sudo k3s ctr images import -

kubectl apply -f infra/spark-conf/spark-rbac.yaml
```

Código A.2: Construcción e importación de las imágenes de la plataforma.

A.4. Ejecución de los flujos

El subconjunto masivo de ORCID exige descargar una sola vez el archivo de resúmenes de perfil, de unos 46 GB, y verificar su suma de comprobación. La tarea de ingesta lo extrae después por su cuenta y nunca lo descarga. Los dos flujos se entregan copiando su fichero al volumen de definiciones de Airflow y se lanzan con la línea de órdenes de Airflow, como muestra el Código A.3.

```
P=$(kubectl get pod -n tfm-lakehouse -l component=dag-processor -o name |
    head -1)
kubectl cp dags/ingest_validate.py \
  tfm-lakehouse/${P#pod/}:/opt/airflow/dags/ingest_validate.py -c dag-
     processor
kubectl exec -n tfm-lakehouse ${P#pod/} -c dag-processor -- \
  airflow dags trigger ingest_validate --run-id run-1 \
  --conf '{"count": 1000, "bulk_max_records": 20000}'

kubectl cp dags/transform_publish.py \
  tfm-lakehouse/${P#pod/}:/opt/airflow/dags/transform_publish.py -c dag-
     processor
kubectl exec -n tfm-lakehouse ${P#pod/} -c dag-processor -- \
  airflow dags trigger transform_publish --run-id run-1
```

Código A.3: Entrega y lanzamiento de los dos flujos de Airflow.

Una ejecución con los parámetros por defecto tarda entre seis y diez minutos con el entorno ya caliente. Los parámetros del flujo de ingesta controlan el número de currículos sintéticos, la semilla, la proporción de currículos enlazados, el tamaño de la muestra de la API y el límite de registros masivos aterrizados. Los del flujo de transformación controlan el número de ejecutores, su memoria, las particiones de mezcla y los umbrales de rechazo y de similitud entre organizaciones.

A.5. Panel de indicadores y verificación

El panel se instala con la imagen propia de Superset, un rol de solo lectura sobre la base de datos de gold y la importación del panel exportado, todo ello descrito en `infra/superset/`

`README.md`. El acceso se hace con una redirección de puerto local. La verificación del conjunto combina la batería de pruebas del repositorio, que se lanza con el comando documentado del Código A.4, y la comprobación del estado real de los pods y de los dos flujos.

```
uv sync --group codegen --group testing
uv run pytest -n auto tests

kubectl get pods -n tfm-lakehouse
kubectl port-forward -n tfm-lakehouse svc/superset 8088:8088
```

Código A.4: Verificación del repositorio y del estado del clúster.

A.6. Recuperación de los fallos conocidos

Dos fallos requieren una intervención manual documentada, ambos recogidos en el registro del Anexo D.8.

- **Reinicio bloqueado del planificador de Airflow.** Una fila de tarea huérfana que dejó un planificador caído hace fallar todos los reinicios posteriores. Se resuelve marcando como fallidas esa tarea y su ejecución desde un pod sano, borrando el pod que nombra si aún existe y reiniciando el pod del planificador.
- **Usuario de PostgreSQL sin crear.** Si la publicación falla con un error de autenticación del usuario de gold tras un arranque en frío, el primer arranque de la base de datos quedó interrumpido. Se confirma revisando el registro del contenedor anterior y se resuelve borrando el pod y su volumen persistente para forzar una inicialización limpia.

Anexo B: Resultados completos del banco de pruebas

Este anexo reúne los resultados medidos de la campaña descrita en el Capítulo 6, de los que el cuerpo del documento solo cita las cifras necesarias para su argumento. Todas las cifras proceden de los registros de ejecución de la campaña, procesados por el módulo de informes del repositorio, y la versión completa, con todas las columnas medidas, está en `docs/benchmark/results.md`.

B.1. Volumen de datos por escala

Cada escala contiene a la anterior, de modo que los registros de ORCID y los currículos sintéticos de la escala menor forman parte de la mayor. La Tabla 7.5 recoge lo que se aterrizó realmente en cada una. La tasa de rechazo se mantiene alrededor del 2,7% en las tres escalas, lo que indica que la composición de los datos no cambia con el volumen.

Escala	Registros masivos de ORCID, con rechazo	CVN sintéticos, con identificador	API	Registros de persona	Tamaño
1x	19 469, 2,66%	11 000, 7 800	200	30 669	2,82 GB
2x	38 920, 2,70%	22 000, 15 500	200	61 120	5,49 GB
4x	77 868, 2,67%	44 000, 31 003	200	122 068	10,94 GB

Tabla 7.5: Datos aterrizados en cada escala del banco de pruebas.

B.2. Tiempos, aceleración y eficiencia por trabajo

Las tres tablas siguientes dan, para cada combinación de escala y número de ejecutores, el número de ejecuciones válidas, la mediana y el rango de tiempo de aplicación, la aceleración respecto a un ejecutor, su eficiencia y el uso medio de procesador del sistema. Una celda vacía indica que esa configuración no produjo ninguna ejecución válida.

Escala	Ejec.	Válidas	Mediana, s	Rango, s	Acel.	Efic.	CPU, %
1x	1	1	337,1	337,1	1,00	1,00	11,4
1x	2	3	224,4	215,5 a 235,5	1,50	0,75	18,2
1x	4	3	206,8	204,3 a 421,0	1,63	0,41	28,1
2x	1	0	–	–	–	–	–
2x	2	1	383,0	383,0	–	–	18,0
2x	4	2	742,1	405,3 a 1 078,9	–	–	23,0
4x	1	0	–	–	–	–	–
4x	2	0	–	–	–	–	–
4x	4	1	1 125,1	1 125,1	–	–	19,8

Tabla 7.6: Validación y normalización de bronze a silver.

Escala	Ejec.	Válidas	Mediana, s	Rango, s	Acel.	Efic.	CPU, %
1x	1	3	78,6	77,4 a 80,5	1,00	1,00	14,8
1x	2	3	74,5	71,4 a 76,4	1,06	0,53	22,7
1x	4	3	85,8	85,1 a 89,5	0,92	0,23	32,2
2x	1	3	115,0	113,5 a 121,3	1,00	1,00	13,1
2x	2	3	102,5	102,4 a 104,3	1,12	0,56	19,0
2x	4	3	109,1	107,5 a 109,5	1,05	0,26	29,3
4x	1	3	302,4	268,1 a 384,4	1,00	1,00	8,7
4x	2	2	280,2	279,5 a 280,9	1,08	0,54	11,2
4x	4	3	257,6	196,7 a 259,0	1,17	0,29	18,5

Tabla 7.7: Cálculo de indicadores de silver a gold.

Escala	Ejec.	Válidas	Mediana, s	Rango, s	Acel.	Efic.	CPU, %
1x	1	3	69,3	62,9 a 72,8	1,00	1,00	8,3
1x	2	3	77,0	74,3 a 77,9	0,90	0,45	11,1
1x	4	3	87,2	83,8 a 88,9	0,79	0,20	15,6
2x	1	3	121,1	116,5 a 137,9	1,00	1,00	8,3
2x	2	3	126,0	117,6 a 128,6	0,96	0,48	8,6
2x	4	3	141,9	136,6 a 164,3	0,85	0,21	11,2
4x	1	3	61,1	60,6 a 61,3	1,00	1,00	11,2
4x	2	3	62,2	58,7 a 62,2	0,98	0,49	13,8
4x	4	3	69,8	68,4 a 71,2	0,88	0,22	19,2

Tabla 7.8: Publicación de gold en PostgreSQL.

Tres rasgos de estas tablas importan para interpretarlas. Primero, la aceleración de silver solo existe a escala base, porque a escalas mayores las configuraciones de uno y dos ejecutores agotan su memoria y no producen ejecuciones válidas con las que calcularla. Segundo, el rango de la configuración de cuatro ejecutores a escala base es muy amplio, con un máximo de 421,0 segundos frente a una mediana de 206,8, y esa dispersión es la razón por la que se repitió cada punto tres veces. Tercero, la publicación a escala cuádruple tarda menos que a escala doble, lo que confirma que su tiempo depende del número de tablas publicadas y no del volumen de filas que contienen.

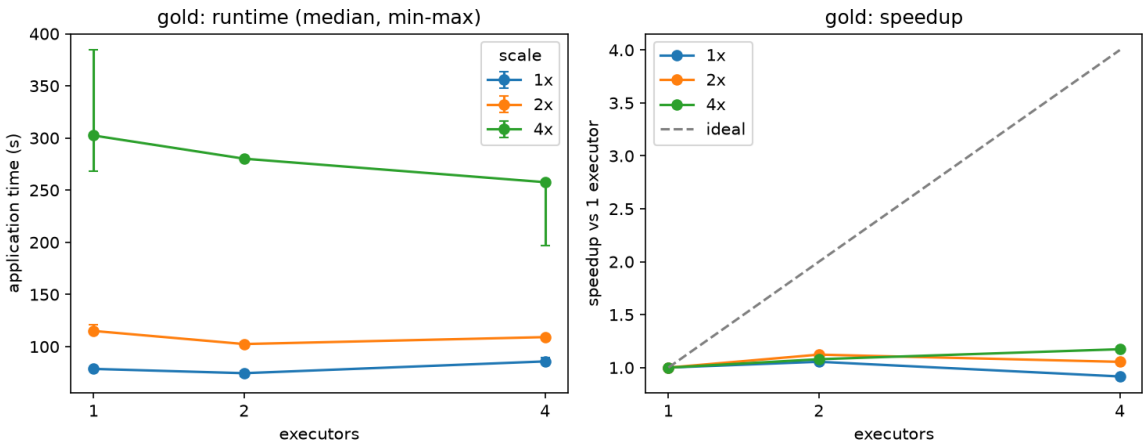


Figura 7.1: Tiempo de ejecución y aceleración del cálculo de indicadores.

B.3. Ejecuciones excluidas

Una ejecución se excluye de las cifras cuando no es comparable con las demás. El informe de la campaña lista cada exclusión con su causa, y todas ellas pertenecen a una de las clases siguientes:

- **Agotamiento de memoria de un ejecutor.** Es el límite determinista descrito en el Capítulo 6, que afecta a un ejecutor desde el doble de volumen y a dos ejecutores a cuádruple volumen. Una vez confirmado en dos intentos sin ningún éxito, la campaña deja de repetir esa configuración.
- **Cierre abrupto de un ejecutor en gold.** Ocurrió una sola vez, con dos ejecutores a cuádruple volumen, y no es determinista.
- **Registro de eventos ilegible.** Afectó a una ejecución de silver con cuatro ejecutores a cuádruple volumen.
- **Ejecutores huérfanos atribuidos a otra ejecución.** Nueve ejecuciones de publicación a cuádruple volumen se marcaron como excluidas por pods ejecutores que pertenecían a una ejecución anterior fallida. Se corrigieron a mano, y cada registro conserva una nota con la prueba de la corrección.

B.4. Corrección del resultado entre configuraciones

Para cada escala y trabajo, las configuraciones que sí produjeron salida dieron la misma huella de contenido, con independencia del número de ejecutores. La Tabla 7.9 resume la comprobación.

Escala	Salida	Ejecutores sin salida	Huellas idénticas
1x y 2x	Gold y publicación	Ninguno	Sí
1x	Silver	Ninguno	Sí
2x	Silver	1 ejecutor	Sí entre los restantes
4x	Gold y publicación	Ninguno	Sí
4x	Silver	1 y 2 ejecutores	No aplicable, solo uno produjo salida

Tabla 7.9: Comprobación de igualdad de contenido entre configuraciones.

Anexo C: Modelo de datos de la plataforma

Este anexo describe las estructuras de datos que atraviesan la plataforma, desde el sobre de procedencia de bronze hasta las tablas publicadas en PostgreSQL. Las definiciones proceden del código del repositorio, de los ficheros `src/tfm_lakehouse/silver/schemas.py` y `src/tfm_lakehouse/gold/schemas.py`, que las declaran como datos y no como tipos de Spark para poder comprobarlas sin ejecutar Spark.

C.1. Organización del almacenamiento

Bronze guarda ficheros de texto en formato JSON Lines bajo el prefijo `bronze/` del cubo `lakehouse`. Cada fuente tiene su propia ruta, y cada ejecución escribe en una partición con su fecha y su identificador, con un fichero de manifiesto que marca la partición como completa:

`bronze/source=<fuente>/ingestion_date=<fecha>/run_id=<ejecución>/part-NNNNN.jsonl`

Los registros que no superan la comprobación de aterrizaje se guardan en una ruta paralela de rechazados con los mismos niveles. Silver y gold son tablas Iceberg bajo la raíz `warehouse/` del mismo cubo, en los espacios de nombres `lakehouse.silver` y `lakehouse.gold`.

C.2. Sobre de procedencia de bronze

Cada línea aterrizada es un sobre con los campos de la Tabla 7.10. El contenido original de la fuente viaja sin modificar en el campo de carga útil, como texto XML para ORCID masivo y como objeto JSON para la API y para el CVN sintético.

Campo	Contenido
`record_id`	Identificador único del registro, base de la deduplicación.
`source`	Fuente de origen: `orcid_bulk`, `orcid_api` o `synthetic_cvn`.
`source_ref`	Referencia dentro de la fuente, como un identificador ORCID o un identificador de documento CVN.
`source_snapshot`	Identificador de la instantánea de origen, que incluye el límite de registros aterrizados.
`source_files`, `source_artifacts`	Ficheros y artefactos de origen, con el vocabulario del bloque de trazabilidad de Open CVN.
`retrieved_at`	Momento en que se obtuvo el dato de su fuente.
`landed_at`, `ingestion_date`	Momento y fecha de la ejecución que lo aterrizó.
`ingestion_run_id`	Identificador de la ejecución, válido como segmento de ruta.
`landing_check`	Estado de la comprobación de aterrizaje y sus errores.
`payload_format`	Formato de la carga útil, `xml` o `json`.
`payload`	Contenido original de la fuente.

Tabla 7.10: Campos del sobre de procedencia de bronze.

C.3. Tablas de silver

Silver tiene seis tablas Iceberg. Todas se reconstruyen por completo en cada ejecución, y dos ejecuciones sobre la misma entrada producen el mismo contenido, incluidos los identificadores de entidad. La Tabla 7.11 resume sus columnas. Las columnas de procedencia de `person_record` y de `rejected` copian las del sobre.

Tabla	Columnas principales
`person_record`	Una fila por registro válido de cualquier fuente: `record_id`, `source`, `source_ref`, nombres y apellidos con sus formas normalizadas, clave de bloqueo, inicial del nombre, `orcid_id`, fecha de última modificación, columnas de procedencia y avisos de validación.
`affiliation`	Una fila por empleo o formación: tipo, organización y su forma normalizada, cargo, departamento, ciudad, país y año y mes de inicio y de fin.
`publication`	Una fila por obra: título y su forma normalizada, año, mes, DOI, tipo de obra y autores, que solo existen en los CVN.
`entity_link`	Una fila por registro: entidad asignada, regla de fusión, evidencia en JSON y los indicadores de ambigüedad y de conflicto de nombre.
`entity`	Una fila por persona resuelta: identificador, identificador ORCID, nombre, número de registros, fuentes, reglas usadas y si tiene CVN y ORCID.
`rejected`	Una fila por registro rechazado: reglas incumplidas, errores y columnas de procedencia.

Tabla 7.11: Tablas de silver.

Los registros rechazados indican siempre la regla concreta que incumplieron. Las reglas son once, seis propias del CVN y cinco de ORCID:

- **CVN:** análisis del JSON, esquema del documento, esquema de cada entidad, dígito de control del identificador ORCID declarado, presencia de los campos obligatorios y orden de las fechas.
- **ORCID:** análisis del XML, forma del JSON de la API, dígito de control del identificador, presencia de los campos obligatorios y existencia de alguna actividad.

Una línea que no es un sobre válido se rechaza antes con una regla propia.

C.4. Tablas de gold y su espejo en PostgreSQL

Gold tiene cinco tablas, escritas en Iceberg y copiadas una a una al esquema `gold` de PostgreSQL. La Tabla 7.12 las describe junto con sus claves y sus índices secundarios. El mapeo de tipos es directo, y solo los instantes de tiempo pasan a un tipo con zona horaria.

Tabla	Contenido	Clave e índices
`dim_researcher`	Una fila por entidad, con nombre, fuentes, número de obras distintas, primer y último año de publicación y rango de años de carrera.	Clave por entidad
`publications_per_researcher_year`	Número de obras distintas por entidad y año.	Clave por entidad y año, índice por año
`affiliation_timeline`	Periodos de afiliación por entidad, con organización, cargo y años.	Índices por entidad y por organización
`collaboration_pairs`	Pares de entidades con obras compartidas, años extremos y marca de miembro con CVN.	Clave por par, índice por segunda entidad
`gold_run`	Una fila por ejecución, con sus recuentos y las instantáneas de silver leídas.	Clave por ejecución

Tabla 7.12: Tablas de gold publicadas en PostgreSQL.

La tabla de ejecución se escribe la última, y la publicación se niega a funcionar si no existe. Su presencia marca así una escritura completa y permite atribuir cada cifra publicada a las instantáneas exactas de silver de las que procede. En la ejecución de referencia las cinco tablas tenían 29 767, 172 861, 94 431, 9 540 y 1 filas, en el orden de la tabla.

Anexo D: Resolución de identidad en detalle

Este anexo amplía la Sección 5.3 con el detalle de las reglas, el análisis de errores que llevó a fijar su diseño final y la sensibilidad de los resultados a sus parámetros. Todas las cifras proceden de la ejecución de 10 000 documentos CVN descrita en el Capítulo 5, evaluada contra el fichero de control que genera el propio generador sintético.

D.1. Normalización previa a la comparación

La comparación no opera sobre los textos originales sino sobre formas normalizadas, que se guardan en silver junto a ellos.

- **Nombres.** Se pliegan sin acentos, en minúsculas y sin puntuación. Las partículas de apellido, como «de», «del» o «la», se omiten al elegir la clave de bloqueo, que combina el primer apellido significativo y la inicial del nombre.
- **Organizaciones.** Se reducen a su conjunto ordenado de palabras significativas, sin las de relleno y sin las direcciones web que algunos perfiles incluyen en el nombre. El orden de las palabras deja de importar y se puede calcular un solape de conjuntos.
- **Obras.** El DOI pierde sus prefijos y pasa a minúsculas, y el título se normaliza igual que los nombres.

D.2. Compatibilidad de nombres

La regla de repliegue acepta una coincidencia de nombre en tres formas, que se clasifican y se guardan en la evidencia de cada enlace.

- **Completa.** Apellido y nombre coinciden tras la normalización.
- **Nombre relajado.** El apellido coincide y el nombre difiere en que una parte es solo una inicial o en que un nombre compuesto aparece abreviado, como «Ana M» frente a «Ana María». Un nombre que pierde una parte entera sigue sin ser compatible.
- **Apellido acortado.** El nombre coincide y un apellido es el primer apellido del otro, porque muchas personas omiten el segundo.

Un nombre que difiere por ambos lados a la vez no es compatible. Esta restricción nació del análisis de errores de la Sección D.5.

D.3. Evidencia guardada en cada enlace

Cada fila de la tabla de enlaces conserva la regla aplicada y una evidencia en JSON con la forma de coincidencia de nombre y las organizaciones compartidas. Dos indicadores marcan los casos dudosos. El de ambigüedad se activa cuando un registro sin identificador tiene más de una entidad candidata, y en ese caso queda como entidad propia. El de conflicto de nombre se activa cuando un CVN cuyo nombre no es compatible con el de ninguno de los registros de ORCID que comparten su identificador se fusiona con ellos. La fusión se mantiene, porque el identificador es la clave autoritativa, y el indicador permite revisarla.

D.4. Umbral de similitud entre organizaciones

La regla de repliegue exige que alguna organización coincida. El umbral por defecto exige igualdad de las formas normalizadas, y se fijó con datos reales y no por preferencia. Un solape de conjuntos de palabras no separa bien los casos. La misma universidad con un sufijo de campus o de facultad puntúa entre 0,40 y 0,60, mientras que dos universidades distintas de la misma ciudad puntúan 0,67. Ningún umbral intermedio acepta lo primero y rechaza lo segundo, y las traducciones de una misma institución entre español e inglés puntúan 0,00 y no se detectan. La Tabla 7.13 muestra el efecto de bajarlo sobre la ejecución de referencia.

Umbral	Fusiones	Correctas	Falsas	Ambiguas
1,0 a 0,6	144	138	6	0
0,5	147	136	11	2
0,34	149	136	13	2

Tabla 7.13: Efecto del umbral de similitud entre organizaciones.

D.5. Análisis de errores y decisión final

La primera medición a gran escala dio una precisión de la regla de repliegue del 90,8%, por debajo del objetivo del 99% fijado antes de medir. Antes de cambiar nada se analizaron los casos, y el análisis encontró dos efectos distintos.

- **Una evaluación demasiado estricta.** Se consideraba evaluable un documento solo si su semilla tenía un registro de ORCID, de modo que una fusión correcta con una entidad anclada por otro CVN que declaraba el mismo identificador contaba como fusión sin contraparte. Se corrigió la definición y los documentos evaluables pasaron de 180 a 186.
- **Una combinación de evidencia débil.** Los 14 falsos positivos compartían exactamente una organización, mientras que las 78 fusiones con dos organizaciones compartidas o más fueron correctas sin excepción. Por tipo de coincidencia, la de nombre completo dio 79 aciertos y 2 errores, la de nombre relajado 47 y 0, la de apellido acortado 12 y 4, y la que relajaba ambas partes a la vez 0 aciertos y 8 errores.

Prohibir relajar ambas partes del nombre a la vez eliminó los 8 errores sin perder ningún acierto, y es la decisión que quedó en el diseño final. El resultado fue una precisión del 95,8% y una cobertura del 74,2%, medida en los 186 documentos evaluables.

D.6. Alternativas más estrictas

Se midieron variantes que exigen más evidencia. Ninguna alcanza el 99% y todas cuestan cobertura, por lo que no se adoptaron por defecto. La evidencia de cada enlace permite aplicarlas en una consulta posterior sin repetir la ejecución. La Tabla 7.14 las compara.

Exigencia adicional	Precisión	Cobertura
Ninguna, diseño adoptado	95,8%	74,2%
Dos organizaciones si el apellido está acortado	98,5%	70,4%
Dos organizaciones si el nombre no es idéntico	98,3%	62,9%
Dos organizaciones en cualquier fusión	100%	41,9%

Tabla 7.14: Variantes más estrictas de la regla de repliegue.

D.7. Cobertura por variante de nombre

El generador sintético escribe el nombre de los documentos sin identificador como nombre exacto o con una de cuatro variantes, y la Tabla 7.15 da la cobertura de la regla en cada una. Los documentos que quedan sin fusionar son sobre todo los que no declaran ninguna afiliación, 47 de los 186 evaluables, para los que la regla no tiene evidencia por diseño. Esa condición fija un techo del 74,7% a la cobertura alcanzable, y la regla alcanza el 99,3% de ese techo.

Variante de nombre	Fusionados	Evaluables
Nombre exacto	43	59
Inicial del nombre	47	60
Apellido en mayúsculas	26	31
Solo el primer apellido	12	19
Sin acentos	10	17

Tabla 7.15: Cobertura de la regla de repliegue por variante de nombre.

D.8. Método de evaluación

El generador escribe, para cada documento, el identificador ORCID de su semilla y el tipo de enlace que se le dio. Con ese fichero se conoce la entidad correcta de cada documento. La evaluación cuenta como evaluable todo documento cuya semilla tiene algún registro en

silver, ya sea de ORCID o de otro CVN que declara el mismo identificador. Una fusión de un documento cuya semilla no está presente cuenta como falsa, porque nada correcto podía fusionarse con él, y es la medida directa de que los registros sin relación permanecen separados. Con la ejecución por defecto de 1 000 documentos solo 16 de 288 tenían contraparte, frente a los 186 de la ejecución de 10 000, porque el solapamiento entre las dos fuentes depende de cuánto de ORCID se aterrizó. Este método mide la resolución contra los datos sintéticos de la plataforma y no contra datos reales anotados, una limitación recogida en el Anexo D.8.

Anexo E: Registro de limitaciones

Este anexo recoge las limitaciones que el desarrollo de la plataforma dejó documentadas, de las que la Sección 7.5 del Capítulo 7 consolida solo las de mayor alcance. El registro mantenido en el repositorio, en `docs/pipeline/known_limitations.md`, es la fuente autoritativa y describe de cada limitación su impacto, su tratamiento actual y su estado. Este anexo las agrupa en cuatro familias y indica, cuando existe, cómo se resolvió o se acotó cada una.

E.1. Infraestructura y cadena de suministro

- **Imágenes de Bitnami congeladas.** Las imágenes de MinIO, de las dos instancias de PostgreSQL y de los subgráficos de Superset se fijan al registro congelado de Bitnami y no reciben parches de seguridad. Se acepta porque el clúster nunca se expone a Internet.
- **Chart de Superset obsoleto.** Sus mantenedores lo han marcado como obsoleto y la vía oficial es ahora un operador de Kubernetes. Se acepta, y el operador queda como trabajo futuro.
- **Superset local y sin endurecer.** Tiene una réplica de cada componente, sin TLS ni inicio de sesión único, y sus metadatos no tienen copia de seguridad. Se acepta para un clúster local.
- **Montajes del nodo e imagen importada a mano.** Los flujos de ingesta y transformación montan código y datos desde el repositorio y contienen la ruta absoluta de la máquina de desarrollo. Es una decisión deliberada que evita reconstruir imágenes al editar código, y limita los flujos a un único nodo.

- **Python 3.10 en el trabajo de silver.** El trabajo corre con el intérprete de la imagen de Spark y no con el 3.14 del repositorio, de modo que el código compartido debe seguir siendo compatible con 3.10.
- **Registro de cliente en la API de ORCID bloqueado.** Un proyecto individual no puede registrar una aplicación cliente. La plataforma usa el nivel anónimo, con un límite diario menor.
- **Propiedades del controlador de Spark sin efecto.** En modo cliente, la configuración de pod del controlador no se aplica y `spark-submit` no avisa. La configuración se fija en el propio pod que lanza el trabajo.
- **Borrado por lotes como permiso aparte.** Un rol que concede borrado pero no borrado de colección deja al controlador de Spark sin poder limpiar sus pods. El rol del repositorio concede ambos.
- **Servidor de API de Airflow cerrado por su sonda de vida.** Bajo carga, la sonda mataba el servidor y las tareas que arrancaban en ese intervalo fallaban. Resuelta con recursos y sondas explícitos.
- **Reinicio bloqueado del planificador de Airflow.** Una fila de tarea huérfana, dejada por un planificador caído, hace fallar todos los reinicios siguientes por un error aún abierto en Airflow. Resuelta operativamente marcando como fallidas esa tarea y su ejecución.
- **Pods ejecutores en estado desconocido.** Un planificador caído puede dejar pods sin limpiar. Se documenta junto con la limitación anterior.
- **Ejecución perdida al activar un flujo.** Activar un flujo con planificación diaria crea de inmediato la ejecución más reciente que se perdió. Es inocuo porque los flujos son idempotentes por identificador de ejecución.
- **Primer arranque de PostgreSQL interrumpido.** Si se interrumpe durante su inicialización, nunca crea el rol de gold y no se recupera solo. Resuelta operativamente borrando el pod y su volumen.
- **Memoria insuficiente de PostgreSQL.** El perfil mínimo del *chart* mataba el contenedor al preparar las tablas de mayor volumen. Resuelta con recursos explícitos.
- **Pruebas de Spark en contenedores.** Con demasiados contenedores a la vez, la batería fallaba por tiempo agotado. Resuelta limitando a cuatro los contenedores simultáneos.

E.2. Alcance de los datos y de la ingesta

- **Subconjunto de ORCID puntual.** Es una instantánea filtrada por país de afiliación, con supuestos de serialización.
- **Muestra acotada del subconjunto.** Por defecto se aterrizan los primeros 20 000 registros en orden de cubo, que ocupan 2,7 GB, y no el subconjunto completo. El parámetro del flujo permite ampliarlo.
- **Datos sintéticos mixtos.** Los CVN sintéticos combinan datos públicos reales de ORCID con datos personales inventados.
- **Cobertura estrecha del generador.** Solo produce cuatro tipos de entrada, que son identidad, experiencia profesional, formación y publicaciones.
- **Extracción parcial en silver.** Solo se extraen esos cuatro tipos. Las demás secciones se validan sin extraerse, solo se traduce el código numérico de país de España y las obras de ORCID no traen autores.
- **Esquema JSON permisivo.** No impone la forma de cada entrada y difiere del importador XML en el tratamiento de las fechas. Se hereda del trabajo anterior.
- **Lectura conjunta de bronze.** Leerlo como un solo conjunto mezcla cargas útiles de texto y de objeto. Cada fuente se lee por su propia ruta.
- **Cifras de carrera como cotas inferiores.** El 18% de las afiliaciones no tiene año de inicio y el 33% no tiene año de fin, y una estancia nunca se extiende hasta hoy.

E.3. Resolución de identidad e indicadores

- **Resolución conservadora.** La regla de nombre y afiliación alcanza una precisión del 95,8% y una cobertura del 74,2%, por debajo del objetivo del 99% de precisión. Una persona con dos identificadores ORCID cuenta como dos entidades.
- **Calidad medida solo contra datos sintéticos.** La verdad de referencia es el identificador de la semilla y solo hay cuatro variantes de nombre, de modo que las cifras son optimistas para currículos reales.
- **Colaboración como cota inferior.** Silver no tiene entidades de coautor, de modo que dos entidades solo se enlazan si declaran el mismo DOI. Los 9 540 pares incluyen 25 que son una misma persona vista dos veces.

- **Deduplicación por tipo de clave.** Una obra con DOI en un registro y sin él en otro cuenta dos veces. El 1,2% de las obras no tiene año útil y queda fuera del indicador por año.
- **Errores de resolución heredados.** Una fusión falsa mezcla las publicaciones de dos personas. Deshacer las 75 fusiones de una sola organización cambia las entidades un 0,25% y los pares de colaboración un 0,58%.
- **Panel sobre agregados parciales.** Deja fuera 443 obras anteriores a 1980 y 16 231 de 94 431 estancias sin año de inicio o fuera del intervalo 1970-2026, y desactiva su caché de datos.

E.4. Escalabilidad y medición

- **Reconstrucción completa.** Silver y gold se recalculan por entero en cada ejecución y PostgreSQL conserva solo la última publicación. Es adecuado para 20 000 registros masivos y no para los 301 763 del subconjunto completo.
- **Techo de memoria con pocos ejecutores.** Con el tamaño de memoria fijo, un ejecutor, y a cuádruple volumen también dos, agotan la memoria del trabajo de silver de forma determinista.
- **Aceleración limitada en un solo nodo.** Más ejecutores no reducen de forma fiable el tiempo, porque el nodo no está limitado por el procesador.

Anexo F: Repositorio del proyecto y guía de uso

El código fuente completo de la plataforma, junto con este documento, está disponible públicamente en el siguiente repositorio, que es el mismo que contiene el Trabajo Fin de Grado sobre el que se construye este trabajo:

`https://github.com/holt00/TFG-open-cvn-schema`

Este anexo describe la organización del repositorio y orienta a quien quiera consultarlo o ejecutarlo, sin repetir los comandos del Anexo 7.6.

Estructura del repositorio

- `src/tfm_lakehouse/`: código propio de la plataforma, con un subpaquete por etapa. `orcid_client` y `orcid_bulk` acceden a las dos fuentes de ORCID, `synthetic_cvn` genera los currículos sintéticos, `bronze` aterriza los registros, `silver` valida, normaliza y resuelve entidades, `gold` calcula los indicadores y genera el SQL de publicación, `spark_jobs` contiene los puntos de entrada de los trabajos de Spark y `benchmark` contiene el banco de pruebas.
- `dags/`: definición de los flujos `ingest_validate` y `transform_publish` de Airflow.
- `infra/`: todo lo necesario para desplegar la plataforma. Los valores de Helm de los servicios están en `helm-values/`, las imágenes y la configuración de Iceberg y de Spark en `spark-conf/` e `ingest/`, y el panel de Superset, su rol de solo lectura y su importación en `superset/`. Cada subdirectorio tiene su propio `README.md`.
- `tests/`: batería automatizada, que incluye las pruebas de los trabajos de Spark ejecutadas dentro de la imagen del clúster.

- `docs/benchmark/`: resultados completos del banco de pruebas, con los registros de cada ejecución en formato CSV y JSON, las gráficas y el procedimiento para repetir la campaña.
- `docs/roadmap/tfm/`: un documento por tarea, con su plan, las decisiones tomadas y las desviaciones respecto a lo planeado.
- `docs/pipeline/known_limitations.md`: registro de limitaciones resumido en el Anexo D.8.
- `docs/memoria/TFM/`: código fuente de este documento.
- `src/open_cvn/`, `src/open_cvn_app/` y `schemas/`: formato Open CVN, herramienta local y esquema JSON del trabajo anterior, de los que la plataforma toma el contrato de validación del currículo.

Cómo orientarse en el repositorio

El punto de entrada es el fichero `README.md` de la raíz, que resume el objetivo y el alcance. El fichero `PROJECT_GUIDE.md` añade un mapa de la documentación bajo `docs/`. Para estudiar el desarrollo en orden cronológico sirve el registro de estado en `docs/context/tfm/current_status.md`, que recoge cada tarea con sus cifras medidas, y para entender una decisión concreta conviene abrir el documento de la tarea correspondiente, cuyas tablas de decisiones enumeran cada alternativa descartada con su motivo.

Para reproducir la plataforma, la guía mantenida es `docs/development/tfm_lakehouse_workflow.md`, que el Anexo 7.6 resume. Para repetir solo la campaña de rendimiento, el procedimiento está en `docs/benchmark/README.md`, y trabaja sobre espacios de nombres aislados que no alteran las tablas de producción.